

Batched and Packed (Publicly) Verifiable Secret Sharing: A Unified Framework and Applications

Shahla Atapoor¹, Karim Baghery¹, Georgio Nicolas¹,
Robi Pedersen², and Jannik Spiessens¹

¹ COSIC, KU Leuven, Leuven, Belgium

² Technical University of Denmark, Denmark

firstname.lastname@kuleuven.be, robi.pedersen@protonmail.com

Abstract. Verifiable Secret Sharing (VSS) allows a dealer to distribute a secret among n parties so that each can verify their share’s validity, and any qualified subset can reconstruct the secret. Publicly Verifiable Secret Sharing (PVSS) extends VSS by enabling anyone to verify the correctness of distributed shares. Both VSS and PVSS schemes are core building blocks in many cryptographic applications. We introduce a *k-batched* and *l-packed* extension of Π , a unified framework from PKC 2025 for Shamir-based computational VSS in the synchronous setting with optimal resilience. Our framework enables the sharing and verification of $l \times k$ secrets in a single protocol execution, offering a tunable trade-off between efficiency and robustness: the *k*-batched, non-packed variant ($l = 1$) improves performance while maintaining optimal resilience, whereas the *k*-batched, *l*-packed variant achieves even greater efficiency at the cost of slightly reduced fault tolerance. Using this framework, we construct several Batched and Packed (BP) VSS and PVSS schemes that significantly reduce both computational and communication costs for the dealer and parties. When sharing many secrets, two of our VSS schemes and our PVSS scheme perform almost as efficiently as plain Shamir sharing. For example, when sharing more than 100 secrets, the overhead of our hash-based BP-VSS is below 3%, for our BP-VSS with information-theoretic privacy it remains around 8%, and for our BP-PVSS it is under 2%. These results show that verifiability in Shamir secret sharing can be achieved in post-quantum and large-scale settings with negligible overhead for the dealer. Our proposed BP-PVSS scheme is the first that can achieve these properties and outperforms existing state-of-the-art protocols. As an application, we show that our BP-PVSS yields substantial performance improvements for the ALBATROSS randomness generation protocol from ASIACRYPT 2020.

Keywords: Verifiable Secret Sharing · Batched and Packed Verifiable Secret Sharing · Batched PVSS · Shamir Secret Sharing

1 Introduction

Secret sharing schemes are fundamental building blocks in modern cryptographic systems that aim to distribute trust among multiple parties. They serve as

core components in secure multi-party computation and threshold cryptography, enabling the construction of distributed protocols without relying on a single trusted entity. These protocols allow a dealer to distribute a secret among a set of participants with the guarantee that only authorized subsets of participants can reconstruct the secret, while unauthorized subsets learn nothing about it. Two well-known traditional secret sharing schemes have been proposed in 1979 by Shamir [35] and Blakley [9]. Shamir’s secret sharing scheme [35] relies on polynomial interpolation over finite fields to divide a secret into parts, with any subset of participants larger than a predetermined threshold able to reconstruct the secret. Blakley’s [9] geometric approach achieves a similar goal using the intersection properties of hyperplanes. Shamir and Blakley focus on securely distributing a secret, but they do not provide mechanisms to verify the integrity and honesty of the shares distributed by the dealer. In other words, while these schemes are robust against passive adversaries, they do not address active adversaries who may distribute incorrect shares during the sharing phase or provide invalid inputs during secret reconstruction.

(Publicly) Verifiable Secret Sharing. Verifiable Secret Sharing (VSS) [2–4, 6, 12, 21, 24, 28, 32, 37] and Publicly Verifiable Secret Sharing (PVSS) [4, 13, 14, 16, 29, 30, 34] protocols were developed to overcome the limitations of classical secret sharing schemes, such as Shamir’s [35], which are secure only against passive adversaries. VSS protocols address this by allowing participants to verify the consistency of their received shares, ensuring that a malicious dealer cannot distribute invalid or inconsistent shares without detection. PVSS schemes further extend this functionality by enabling anyone, including external observers, to verify the correctness of the shared values, making them especially useful in publicly auditable settings. VSS protocols can be classified by their security guarantees and communication assumptions. In terms of adversarial models, VSS schemes can be either Information-Theoretically (IT) secure, offering unconditional guarantees, or computationally secure, relying on the hardness of cryptographic assumptions. The communication setting also significantly affects protocol design: in synchronous networks, where message delays are bounded, computationally secure VSS schemes can tolerate up to $t < n/2$ corrupted parties. In contrast, asynchronous networks, where message delays are unbounded, typically require a greater fraction of honest participants to ensure correctness and security. The recent demand for highly efficient and Post-Quantum (PQ) secure VSS schemes has spurred the development of hash-based (lightweight) Shamir-based protocols [2, 4, 12, 38]. These modern VSS schemes aim to minimize computational and communication overhead while maintaining (PQ) security against malicious parties. Building upon the ideas from [2], Bagheri recently proposed a unified framework called Π to build Shamir-based computationally secure VSS schemes in the synchronous communication model that can achieve optimal resilience.

The Need for Batched and Packed (P)VSS Protocols. VSS and PVSS are fundamental building blocks in distributed cryptographic protocols including threshold cryptosystems, Distributed Key Generation (DKG), threshold signatures, secure Multi-Party Computation (MPC), distributed randomness genera-

tion, and privacy-preserving federated learning [2, 14, 22, 23, 27, 39]. Despite their foundational role, these protocols often encounter efficiency bottlenecks in practice, particularly in cases that require sharing a large number of secrets.

These challenges motivate the development of Batched VSS (BVSS) protocols. These protocols aim to amortize the computational and communication costs across multiple VSS instances while preserving optimal resilience. A k -BVSS protocol enables a dealer to share k secrets with the same set of participants in a single protocol execution without compromising the protocol's resilience. Functionally, this is equivalent to running Shamir's protocol k times, once per secret, with the same (n, t) parameters, but generating a single proof to prove the correctness of all $n \times k$ distributed shares. This approach can significantly improve both computational and communication efficiency compared to executing k separate VSS protocols, once per secret. BVSS is particularly well-suited for applications that involve repeated sharing among a fixed group of parties, offering performance gains without sacrificing robustness.

An alternative approach is offered by *packed secret sharing schemes* [10, 26], which improve the efficiency of Shamir secret sharing by encoding ℓ secrets into a single polynomial, without increasing the share size. Standard Shamir sharing is a special case where $\ell = 1$. However, since at most $t+1$ independent secrets can be securely embedded in a degree- t polynomial, packing ℓ secrets requires increasing the degree to $t + \ell$ and ensuring that the number of parties satisfies $n \geq 2t + \ell$ in the synchronous setting. As a result, increasing the packing factor ℓ typically reduces the tolerable corruption threshold t , weakening the scheme's robustness. In essence, packed secret sharing trades resilience for efficiency: it provides t -privacy and $(t + \ell)$ -reconstruction, but does not achieve optimal resilience.

Given the complementary strengths and limitations of the batched and packed approaches, a compelling research question emerges:

Can we design practical, tunable VSS or PVSS schemes that support both batching and packing, thereby unifying their efficiency and robustness benefits while preserving verifiability?

Our Contributions. Our contributions can be summarized as follows:

A Unified Tunable Framework for Batched and Packed VSS. Our primary contribution is proposing a tunable variant of the Π framework, originally introduced by Bagheri [4], designed to support both *batched* and *packed* secret sharing. The original Π framework facilitates the construction of Shamir-based, computationally secure VSS schemes in the synchronous communication model with optimal resilience. Our new variant, denoted (k, ℓ) -BII (or simply BII), retains the core requirements of the original, namely, a random oracle and a secure (not necessarily homomorphic) vector commitment scheme. However, (k, ℓ) -BII distinguishes itself by enabling the simultaneous sharing and verification of $k \times \ell$ secrets in a single protocol execution. Figure 1 visualizes how $k \times \ell$ secrets and $n \times k$ shares are encoded in (k, ℓ) -BII via k independent random polynomials of degree $t + \ell - 1$. Each of the ℓ secrets is encoded at points $j = 0, -1, -2, \dots, -(\ell - 1)$ of a degree- $(t + \ell - 1)$ polynomial. The dealer then

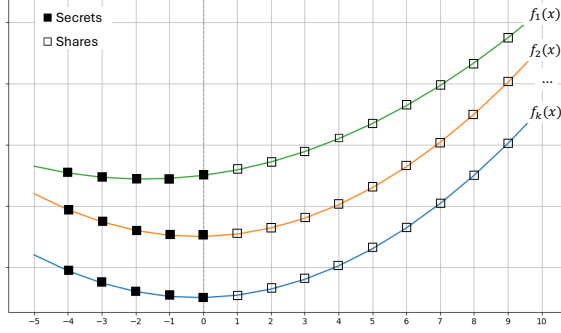


Fig. 1. A visualization of $k \times \ell$ secrets (■) and $k \times n$ shares (□) encoded in (k, ℓ) -BII via k random degree $t + \ell - 1$ polynomials. The $k \times \ell$ secrets are encoded at points $j = 0, -1, \dots, -(\ell - 1)$, while the $k \times n$ shares are encoded at points $i = 1, 2, \dots, n$. A dealer produces a single threshold proof to prove the correctness of all $k \times n$ shares.

generates a single threshold proof to prove the validity of all $k \times n$ shares, which are encoded at points $i = 1, 2, \dots, n$ of the k random polynomials. The BII framework can yield substantial efficiency gains over executing the original Π protocol separately for each secret. Specifically, when sharing k secrets among n participants, the $(k, 1)$ -BII variant reduces the total proof size from $O(nk\lambda)$ to $O((n + k)\lambda)$, without sacrificing optimal resilience, where λ denotes the security parameter. In addition to reduced communication, the computational load for both the dealer and participants is significantly decreased. Further performance improvements can be achieved by setting $\ell > 1$, which allows ℓ secrets to be packed into a single polynomial. However, this comes at the cost of lowering the protocol’s resilience, as the number of tolerable corruptions must be reduced. The tunable nature of BII makes it a versatile framework for building a wide range of computationally secure VSS schemes in the synchronous setting, supporting batching, packing, or both.

Practical Batched and Packed VSS (BP-VSS) Schemes. Building on the BII framework and employing various vector commitment schemes, we construct a suite of practical, computationally secure Batched and Packed VSS (BP-VSS) protocols: namely, BII_{LA} , BII_{P} , $\text{BII}_{\text{P}+}$, and BII_{F} . Each instantiation offers distinct features, security guarantees, and efficiency trade-offs, tailored to different application needs. More concretely, we instantiate BII using both discrete logarithm- and hash-based vector commitments to derive new BP-VSS protocols. The new BP-VSS schemes serve as batched and packed counterparts to the original VSS schemes Π_{LA} , Π_{P} , and Π_{F} from [4].

BII_{F} is the batched and packed variant of Π_{F} , an alternative to Feldman’s VSS [24]. Like Feldman’s scheme, BII_{F} requires high-entropy secrets and relies on the hardness of the Discrete Logarithm (DL) problem.

BII_{P} is the batched and packed analogue of Π_{P} [4]. Much like Pedersen’s VSS [32], it supports both high- and low-entropy secrets while achieving perfect simulatability, ensuring that even a computationally unbounded adversary with

access to t shares learns nothing about the secrets. To further improve BII_P and construct BII_P+ , we introduce a novel perfectly hiding (basic) vector commitment scheme that achieves substantially better performance than the commonly used additively homomorphic Pedersen vector commitment. Our scheme requires only two exponentiations and one hash evaluation, in contrast to the $n + 1$ exponentiations needed in the vector variant of Pedersen commitment. We believe this commitment scheme may be of independent interest in applications requiring non-homomorphic and perfectly hiding commitments. In particular, it can also enhance the efficiency of the original Π_P VSS protocol [4] in both the sharing and verification phases.

Lastly, BII_{LA} represents a batched and packed variant of the hash-based VSS scheme Π_{LA} [4], and offers an optimized and batched alternative to the ABCP VSS protocol [2]. BII_{LA} also serves as a synchronous and packed alternative to the asynchronous Shoup-Smart VSS protocol [38], operating under the milder assumption of an honest majority rather than the stricter $2/3$ bound.

Batched and Packed Variants of Pedersen and Feldman Schemes. As an additional contribution, we introduce batched and packed version of the original Feldman and Pedersen VSS schemes [24, 32]. These protocols achieve significant efficiency gains over k sequential executions of their base protocols: broadcast communication is reduced by the factor k , and verification cost is lowered from $\mathcal{O}(kn)$ to $\mathcal{O}(k + n)$ exponentiations, where n denotes the number of parties and k the batching parameter. While their verification is less efficient compared to our BII -based schemes, they offer the advantage of operating in the plain model.

Efficiency of New BP-VSS Schemes. Table 1 summarizes the asymptotic efficiency of our new schemes, reporting both communication and computation costs, and highlighting natural groupings of constructions that achieve comparable security guarantees. For benchmarking, we compare these results against k concurrent executions of the packed variants of the original protocols from [4, 24, 32]. To ensure fairness, we first computed the packed costs of the original schemes (even though these were not explicitly presented in [4, 24, 32]).

An Efficient Batched and Packed PVSS Scheme. While our proposed BP-VSS schemes, BII_F , BII_P , BII_P+ , and BII_{LA} , support only designated verification (i.e., correctness of distributed shares can only be verified by a subset of the respective recipients), our next contribution addresses the need for public verifiability. We introduce a Batched and Packed PVSS scheme, denoted BII_S , built upon the BII_F BP-VSS scheme. This new construction supports both batched and packed secret sharing with public verifiability. To the best of our knowledge, BII_S is the first PVSS scheme that enables both batching and packing of secrets.

To construct BII_S , we first develop a batched version of the generalized Schnorr protocol introduced by Bagheri [4], which allows a prover to demonstrate knowledge of multiple witness polynomials in the Polynomial Discrete Logarithm (PDL) relation defined therein. We refer to this new batched sigma protocol as π_{BPDL} . In π_{BPDL} , a prover can prove knowledge of k secret polynomials at nearly the same computational and communication cost as proving a

Table 1. Performance of the proposed Batched and Packed VSS (BP-VSS) schemes for sharing $k \times \ell$ secrets, compared with k concurrent executions of the packed variants of the original protocols from [4, 24, 32]. Abbreviations: Comm.: Communication, (q)RO: (quantum) Random Oracle, Plain: Plain Model, BC: Broadcast, n : Number of parties, t : number of malicious parties ($t \approx \frac{n-\ell}{2}$), k : batching parameter, ℓ : number of secrets encoded in each function, $E_{\mathbb{G}}$: Exponentiation in group $\mathbb{G}$, $M_{\mathbb{G}}$: Multiplication in group $\mathbb{G}$, $M_{\mathbb{F}}$: Multiplication in finite field $\mathbb{F}$, $\mathcal{PE}$: degree- d Polynomial Evaluation with $d = t + \ell - 1$, $\mathcal{H}$: Hashing, $|X|$: X element size, $|\mathcal{H}|$: Output size of $\mathcal{H}$.

BP-VSS & Model	Sharing	Dealer's Comm.	Verification	Reconstruct
k runs of packed Fel [24], plain	$dk E_{\mathbb{G}}$ $nk \mathcal{PE}$	Private: $1nk \mathbb{Z}_q $ BC: $dk \mathbb{G} $	$dk + k E_{\mathbb{G}} + dk M_{\mathbb{G}}$	$d^2k + dk E_{\mathbb{G}}$ $d^2k M_{\mathbb{G}}$
BP-Feldman Sec. 4.5, plain	$dk E_{\mathbb{G}}$ $nk \mathcal{PE}$	Private: $1nk \mathbb{Z}_q $ BC: $d \mathbb{G} $	$d + k E_{\mathbb{G}} + d + k M_{\mathbb{G}}$	$d^2 + dk E_{\mathbb{G}} + d^2 + dk M_{\mathbb{G}}$
k runs of packed Π_F [4], RO	$2nk E_{\mathbb{G}}$ $2nk \mathcal{PE}$	Private: $1nk \mathbb{Z}_q $ BC: $nk \mathbb{G} + dk \mathbb{Z}_q $	$2k E_{\mathbb{G}} + k \mathcal{PE} + k \mathcal{H}$	$2dk E_{\mathbb{G}} + dk \mathcal{PE} + dk \mathcal{H}$
BII _F , Sec. 4.4 RO	$nk + n E_{\mathbb{G}}$ $nk + n \mathcal{PE}$	Private: $1nk \mathbb{Z}_q $ BC: $n \mathbb{G} + d \mathbb{Z}_q $	$k E_{\mathbb{G}} + 1 \mathcal{PE} + 1 \mathcal{H}$	$dk E_{\mathbb{G}} + d \mathcal{PE} + d \mathcal{H}$
k runs of packed Ped [32], plain	$dk + d E_{\mathbb{G}}$ $nk + n \mathcal{PE}$	Private: $2nk \mathbb{Z}_q $ BC: $dk \mathbb{G} $	$dk + k E_{\mathbb{G}} + kd M_{\mathbb{G}}$	$d^2k + dk E_{\mathbb{G}} + kd^2 M_{\mathbb{G}}$
BP-Pedersen Sec. 4.5, plain	$dk + d E_{\mathbb{G}}$ $nk + n \mathcal{PE}$	Private: $2nk \mathbb{Z}_q $ BC: $d \mathbb{G} $	$d + k E_{\mathbb{G}} + d + k M_{\mathbb{G}}$	$d^2 + dk E_{\mathbb{G}} + d^2 + dk M_{\mathbb{G}}$
k runs of packed Π_P [4], RO	$3nk E_{\mathbb{G}}$ $2nk \mathcal{PE}$	Private: $2nk \mathbb{Z}_q $ BC: $nk \mathbb{G} + dk \mathbb{Z}_q $	$3k E_{\mathbb{G}} + k \mathcal{PE} + k \mathcal{H}$	$3dk E_{\mathbb{G}} + dk \mathcal{PE} + dk \mathcal{H}$
BII _P , Sec. 4.2 RO	$nk + 2n E_{\mathbb{G}}$ $nk + n \mathcal{PE}$	Private: $2nk \mathbb{Z}_q $ BC: $n \mathbb{G} + d \mathbb{Z}_q $	$k E_{\mathbb{G}} + 1 \mathcal{PE} + 1 \mathcal{H} + 2K M_{\mathbb{G}}$	$dk E_{\mathbb{G}} + d \mathcal{PE} + d \mathcal{H} + 2dK M_{\mathbb{G}}$
BII _P ⁺ , Sec. 4.2 RO	$3n E_{\mathbb{G}} + n \mathcal{H}$ $nk + n \mathcal{PE}$	Private: $2kn \mathbb{Z}_q $ BC: $n \mathbb{G} + d \mathbb{Z}_q $	$2 E_{\mathbb{G}} + 1 \mathcal{PE} + 2 \mathcal{H} + k M_{\mathbb{G}}$	$2d E_{\mathbb{G}} + d \mathcal{PE} + 2d \mathcal{H} + dk M_{\mathbb{G}}$
k runs of packed Π_{LA} [4] (q)RO	$1nk \mathcal{H}$ $2nk \mathcal{PE}$	Private: $1nk \mathbb{Z}_N $ BC: $nk \mathcal{H} + dk \mathbb{Z}_N $	$k \mathcal{PE} + 2k \mathcal{H}$	$dk \mathcal{PE} + 2dk \mathcal{H}$
BII _{LA} , Sec. 4.1 (q)RO	$1n \mathcal{H}$ $nk + n \mathcal{PE}$	Private: $1nk \mathbb{Z}_N $ BC: $n \mathcal{H} + d \mathbb{Z}_N $	$1 \mathcal{PE} + 2 \mathcal{H} + k M_{\mathbb{F}}$	$d \mathcal{PE} + 2d \mathcal{H} + dk M_{\mathbb{F}}$

single one in the original protocol [4]. We believe the π_{BPDL} protocol may be of independent interest. Table 2 summarizes the performance of the new BP-PVSS scheme BII_S and compares it to the case of running k concurrent executions of the packed PVSS protocol Π_S [4, 14] and the packed YOLO-YOSO [17] PVSS protocol to share $k \times \ell$ secrets. As can be seen from Table 2, our proposed scheme BII_S outperforms the state-of-the-art schemes across all evaluated metrics. Notably, compared to Π_S , it achieves approximately a 50% improvement in the computational cost of both the sharing and verification phases. Furthermore, thanks to the batched proof π_{BPDL} , the overall proof size is reduced by a factor of k , significantly decreasing both the communication overhead and the verifier's workload. In addition, BII_S outperforms YOLO-YOSO in sharing, verification, and reconstruction, while also providing a smaller amortized proof size whenever $d < 2k$, where d is the degree of underlying secret polynomial. We also highlight that in a DL-based PVSS scheme even with a trusted dealer, the dealer must run

Table 2. Comparison of the proposed BP-PVSS scheme Π_S for sharing $k \times \ell$ secrets with k concurrent executions of the packed PVSS scheme Π_S and YOLO-YOSO [4, 14, 17]. Abbreviations: PDL: Polynomial Discrete Logarithm, DDH: Decisional Diffie-Hellman, RO: Random Oracle, n : Number of parties, t : number of malicious parties ($t \approx \frac{n-\ell}{2}$), k : batching parameter, ℓ : number of secrets encoded in each polynomial, E_G : Exponentiation in group G , M_G : Multiplication in group G , $\mathcal{PE}$: degree- d Polynomial Evaluation with $d = t + \ell - 1$, $\mathcal{H}$: Hashing, $|X|$: X element size, $|\mathcal{H}|$: Output size of $\mathcal{H}$.

BPVSS & Security	Sharing	Dealer's Comm.	Verification	Reconstruct
k times packed YOLO [17] (DDH, RO)	$4nk E_G$ $2nk \mathcal{PE}$	Cipher: $nk G $ Proof: $2k \mathbb{Z}_q $	$2nk E_G +$ $nk \mathcal{PE} + 2nk M_G$	$5dk E_G +$ $2dk M_G$
k times packed Π_S [4, 14] (PDL, DDH, RO)	$2nk E_G$ $2nk \mathcal{PE}$	Cipher: $nk G $ Proof: $dk \mathbb{Z}_q $	$2nk E_G +$ $nk \mathcal{PE} + nk M_G$	$5dk E_G +$ $dk M_G$
Π_S , Sec. 5.2 (PDL, DDH, RO)	$nk + n E_G$ $nk + n \mathcal{PE}$	Cipher: $nk G $ Proof: $d \mathbb{Z}_q $	$nk + n E_G +$ $n \mathcal{PE} + nk M_G$	$3dk E_G +$ $dk M_G$

the Shamir scheme k times, requiring nk polynomial evaluations, and perform at least nk encryptions to deliver k ciphertexts to each of the n parties. Thus, even in the trusted setting, the baseline cost is nk exponentiations plus nk polynomial evaluations, which serves as the reference point for efficiency comparisons. Relative to this baseline, Π_S achieves nearly identical efficiency.

Implementation Results. To evaluate the practical efficiency of our proposed schemes, we developed a prototype implementation in Rust (see Sec. 6 for details). The results demonstrate that the new BP-VSS and BP-PVSS protocols are state-of-the-art constructions, offering high efficiency in both computation and communication, and are well-suited for deployment in large-scale cryptographic applications. In particular, when sharing a large number of secrets, the BP-VSS schemes Π_{LA} and Π_P+ and the BP-PVSS scheme Π_S achieve performance almost identical to plain Shamir sharing in the distribution phase. For example, when sharing more than 100 secrets, compared to the plain Shamir, the overhead of our hash-based BP-VSS is below 3%, for our BP-VSS with information-theoretic privacy it remains around 8%, and for our BP-PVSS it is under 2%. These findings confirm that verifiability in Shamir secret sharing can be attained in post-quantum and large-scale settings with negligible overhead for the dealer.

ALBATROSS+: Improving Performance of ALBATROSS with Π_S . As our final contribution, and as a practical application of Π_S , we introduce ALBATROSS+, an improved version of the ALBATROSS distributed randomness generation protocol [14], ranking among the most efficient PVSS-based constructions [31]. The core innovation lies in replacing the original protocol's ℓ -packed Π_S PVSS with our proposed Π_S , a k -batched and ℓ -packed version of Π_S . In terms of efficiency, ALBATROSS+ outperforms the original in both computational and communication costs. Notably, it reduces the amortized proof size in protocol from $O(k)$ to $O(1)$, where $k = \frac{L}{(n-2t)^2}$ represents the repetition count in the original protocol, and L denotes the total number of random values that n parties (where up to t of them can be malicious) collectively generate.

These improvements preserve the security guarantees of the original protocol, while significantly boosting its efficiency, enabling practical high-throughput randomness generation with even few participants.

2 Preliminaries

Due to space constraints, we mainly recall verifiable secret sharing in this section and provide a concise overview of the Schwartz-Zippel Lemma, vector commitment schemes and Sigma-protocols along with some relevant constructions in App. A.

Notation. We let λ denote a security parameter. A function is called *negligible in x* , written $\text{negl}(x)$, if for any constant c , there exists some x_0 , such that $f(x) < x^{-c}$ for $x > x_0$. A function that is negligible in the security parameter λ is simply called negligible. We use the assignment operator $\leftarrow$ to denote uniform sampling from a set S , e.g. $y \leftarrow S$. We write $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ with known prime q , and $\mathbb{Z}_q[x]_t$ for polynomials of degree t in the variable x and with coefficients in $\mathbb{Z}_q$. For $n \in \mathbb{N}$, we write $[n] = \{1, \dots, n\}$. Finally, all logarithms are in base 2.

2.1 Shamir, Packed, Batched, and Verifiable Secret Sharing

Shamir Secret Sharing. A $(n, t+1)$ -Shamir secret sharing scheme [36] allows a secret f_0 to be distributed among n parties such that any subset of at most t parties learn no information about the secret, while any subset of at least $t+1$ parties can efficiently reconstruct it. This is achieved via polynomial interpolation over the finite field $\mathbb{Z}_q$. Specifically, a random polynomial $f(x) \in \mathbb{Z}_q[x]_t$ of degree at most t is sampled, with its free term set as the secret, i.e., $f(0) = f_0$. Each party P_i for $i \in \{1, \dots, n\}$ is assigned the secret share $f_i = f(i)$. Then, any subset $Q \subseteq \{1, \dots, n\}$ of size at least $t+1$ parties can reconstruct f_0 using Lagrange interpolation: $f_0 = f(0) = \sum_{i \in Q} f_i \cdot L_{0,i}^Q$, where

$$L_{0,i}^Q := \prod_{j \in Q \setminus \{i\}} \frac{j}{j-i} \pmod{q},$$

are the Lagrange basis coefficients evaluated at 0. Note that in this setting $n \geq 2t+1$ ensures both t -privacy, i.e. any set of up to t parties cannot learn anything about the secret, and reconstruction by at least $t+1$ parties. Further, we can achieve optimal resilience.

Packed Shamir Secret Sharing. Packed Shamir secret sharing [10, 15, 26] generalizes the classical Shamir scheme by allowing multiple secrets to be shared simultaneously using a single polynomial, without increasing the size of each share. This is achieved by encoding ℓ secrets into a single polynomial over $\mathbb{Z}_q$ evaluated at ℓ publicly known positions. Let $f_0, f_{-1}, \dots, f_{-(\ell-1)} \in \mathbb{Z}_q$ denote the ℓ secrets to be shared. A random polynomial $f(x) \in \mathbb{Z}_q[x]$ of degree $d = t + \ell$ is

sampled such that: $f(j) = f_j$ for $j = 0, -1, \dots, -(\ell - 1)$, where t denotes the number of malicious parties the scheme tolerates. Each party P_j for $j = 1, \dots, n$ receives $f_j = f(j)$ as its share. Given any $t + \ell$ valid shares, the entire polynomial $f(x)$, and hence all ℓ secrets, can be uniquely reconstructed via interpolation.

Although the packed variant offers substantial space efficiency by embedding multiple secrets into one polynomial (keeping share sizes constant), it comes with a trade-off in fault tolerance. Specifically, it requires $n \geq 2t + \ell$, and the degree of the polynomial increases with ℓ , thereby reducing the number of tolerated corruptions for a fixed n . Thus, the packed Shamir achieves t -privacy and $(t + \ell)$ -reconstruction, sacrificing optimal resilience for improved efficiency. We also note that the evaluation points used for secrets $(0, -1, \dots, -(\ell - 1))$ and shares $(1, \dots, n)$ may be replaced by any set of $n + \ell$ pairwise distinct points. Alternative choices may lead to more efficient algorithms for both share computation and secret reconstruction [40].

Batched Verifiable Secret Sharing. Standard secret sharing schemes are typically secure only against passive adversaries. However, many applications require security even in the presence of malicious behaviour by the dealer or parties, which is addressed by Verifiable Secret Sharing (VSS) schemes [21]. VSS schemes enable a dealer to distribute a secret among a group of parties in a way that allows each party to verify the correctness of their share, ensuring that only a qualified subset can later reconstruct the secret.

In numerous real-world scenarios, it is either necessary or beneficial to run multiple VSS protocols simultaneously in a batched fashion. That is, a dealer holding k independent secrets can share them in parallel using k separate degree- t polynomials, where t denotes the number of malicious parties the system can tolerate. Importantly, unlike packed secret sharing, this approach preserves optimal resilience: the sharing of each secret is independent, so fault tolerance does not degrade with the number of secrets shared. Below, we summarize the formal notion of Batched VSS (BVSS) schemes, a natural generalization of classical VSS protocols, as proposed in works such as [2, 4, 32].

Definition 1. *An (n, t) -BVSS consists of four PPT algorithms of (Initial, Share, Verify, Reconstruct) as follows:*

1. *Initial: In this phase, public parameters are sampled and shared with the participants.*
2. *Share($n, t, (f_0^{(1)}, \dots, f_0^{(k)})$) $\rightarrow (\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^n, \pi_{share})$: Given (n, t) , k secrets, it outputs the shares $\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^n$, and a (non-interactive) threshold proof π_{share} to prove the validity of the shares.*
3. *Verify($n, t, \{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^n, \pi_{share}$) $\rightarrow \text{true/false}$: Given (n, t) , the shares $\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^n$, and a threshold proof π_{share} , generated by Share, it outputs either **true** or **false**.*
4. *Reconstruct($\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i \in Q, |Q| \geq t+1}$) $\rightarrow \{f_0^{(1)}, \dots, f_0^{(k)}, \text{false}\}$: Given any at least $t + 1$ set of k shares, e.g., $\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^{t+1}$, it either reconstructs and returns the secrets $(f_0^{(1)}, \dots, f_0^{(k)})$, or it returns **false**.*

A BVSS further requires satisfying the following properties.

- **Completeness:** If the dealer honestly executes the **Share** algorithm, then the **Verify** algorithm will always return **true**. Moreover, any subset of at least $t+1$ honest parties will be able to reconstruct the shared secrets $(f_0^{(1)}, \dots, f_0^{(k)})$.
- **Verifiability constraint:** A shareholder must be able to verify the validity of the received shares. If the verification returns **true**, then **Reconstruction** should produce *unique secrets* $(f_0^{(1)}, \dots, f_0^{(k)})$ when run on any $t+1$ distinct valid shares. More formally, for any integers $n \geq 2t+1$ and $t \geq 0$, a BVSS is called verifiable if for any PPT adversaries $\mathcal{A}$, and for all λ , we have

$$\Pr \left[\begin{array}{l} (\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^n, \pi_{share}) \leftarrow \mathcal{A}(n, t), \\ \text{Verify}(n, t, \{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^n, \pi_{share}) = \text{true} : \\ \exists (f_0^{(1)}, \dots, f_0^{(k)}) \forall V \in [n], |V| \geq t+1, \\ \text{Reconstruct}(\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i \in V}, \pi_{share}) \rightarrow (f_0^{(1)}, \dots, f_0^{(k)}) \end{array} \right] \geq 1 - \text{negl}(\lambda),$$

where V is the set of honest parties.

- **Unpredictability:** The protocol must be unpredictable, meaning that there is no strategy for selecting t sets of shares of the secrets that would enable someone to predict the secrets $(f_0^{(1)}, \dots, f_0^{(k)})$ with a significant advantage. More formally, for any integers $n > 1$ and $t < n$, a BVSS is called unpredictable if for all PPT adversaries $\mathcal{A}$ who have non-adaptively corrupted up to t parties, for all λ , and random secrets $(f_0^{(1)}, \dots, f_0^{(k)})$, we have:

$$\Pr \left[\begin{array}{l} (\{f_i^{(1)}, \dots, f_i^{(k)}\}_{i=1}^n, \pi_{share}) \leftarrow \text{Share}(n, t, (f_0^{(1)}, \dots, f_0^{(k)})); \\ Q \subset [n]; |Q| \leq t; (f_0'^{(1)}, \dots, f_0'^{(k)}) \leftarrow \mathcal{A}(n, t, \{f_i^{(1)}, \dots, \\ f_i^{(k)}\}_{i \in Q}, \pi_{share}) : f_0'^{(1)} = f_0^{(1)} \vee \dots \vee f_0'^{(k)} = f_0^{(k)} \end{array} \right] \leq \text{negl}(\lambda).$$

- **Simulatability:** In some cases, the definition of unpredictability can be strengthened by requiring that the view of the adversary who can corrupt up to t parties can be simulated.

We note that by setting $k = 1$, one can obtain the definitions for a typical VSS scheme [2, 4, 32, 34], where the dealer shares a single secret $f_0^{(1)}$.

3 A Unified Framework for Batched and Packed VSS

In this section, we introduce a tunable extension of the Π framework [4], referred to as (k, ℓ) -B Π (or simply B Π), which is designed to support batched and/or packed secret sharing in the synchronous setting. This generalization offers greater flexibility in terms of resilience and efficiently sharing multiple secrets, either by batching several independent sharings, packing multiple secrets into a single polynomial, or combining both approaches. Before presenting the details of the (k, ℓ) -B Π protocol, we refer the reader to App. B for a brief overview of the original Π scheme, which corresponds to the special case $(1, 1)$ -B Π .

3.1 (k, ℓ) -BII: k -Batched and ℓ -Packed Versions of Π

Intuitively, the Π framework can be seen as performing Shamir secret sharing twice, once to share the actual secret and once to prove its correctness, effectively sacrificing one sharing to verify the other. More generally, by embedding auxiliary randomness γ_i into the commitments, Π also supports the sharing of low-entropy secrets. Our investigation reveals that when a dealer wishes to share k independent secrets with the same group of participants, the proof π_{share} in Π can be efficiently batched. Moreover, the protocol can be adapted to support ℓ -packed secret sharing with minimal modifications. However, in the case of packing, we sacrifice the robustness of the protocol. On the positive side, packing can effectively double the number of shared secrets by only requiring one extra honest participant. In this k -batched and ℓ -packed setting, the dealer requires only a single additional polynomial to prove the correctness of k simultaneous ℓ -packed Shamir sharings. As a result, the cost of proving k parallel sharings becomes nearly equivalent to that of a single (packed) sharing, significantly reducing the overall overhead. In contrast to the original Π protocol, where each sharing is handled separately, BII allows the dealer to commit to k shares per participant simultaneously. Moreover, instead of using an individual masking polynomial per secret, the dealer uses a single randomizer polynomial to mask all k secret-sharing polynomials collectively when computing the polynomial $z(x)$. The general construction of (k, ℓ) -BII is illustrated in Fig. 2. It relies on a perfectly or computationally hiding vector commitment scheme $\mathcal{C}$, and the random oracle $\mathcal{RO} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ as a black-box component.

3.2 Security and Efficiency

Security. We prove the security of (k, ℓ) -BII in the following theorem.

Theorem 1 (k -Batched and ℓ -Packed Computational VSS). *If the commitment scheme $\mathcal{C}$ is computationally (resp. perfectly) hiding and perfectly (resp. computationally) binding, and $\mathcal{H}$ is a random oracle, then the generic construction given in Fig. 2 is a secure batched (and packed) VSS scheme in the synchronous setting for $n \geq 2t + \ell$. Specifically: (i) An honest dealer will convince honest shareholders, and any subset of $t + \ell$ honest shareholders will be able to reconstruct the secrets. (ii) The reconstruction protocol outputs a unique set of secrets distributed by the dealer for any qualified set of shareholders (i.e., of size at least $t + \ell$); (iii) Any set of up to t shareholders (i.e., a non-qualified set) cannot obtain any information about the secrets.*

Proof. The proof is given in App. C.1.

Efficiency of (k, ℓ) -BII. The (k, ℓ) -BII protocol significantly improves efficiency over the original Π (equivalent to $(1, 1)$ -BII) when proving the validity of $k \times \ell$ secrets. During the sharing phase, the dealer computes evaluations for k independent degree- $(t + \ell)$ polynomials $f^{(1)}(x), \dots, f^{(k)}(x)$, each requiring n evaluations,

Initial: Parties $P_1, \dots, P_n$ register a PK to facilitate secure communications and agree on public parameters, like random oracle $\mathcal{RO}$ and commitment scheme $\mathcal{C}$. Let there exist a dealer D and $P_1, \dots, P_n$ parties who will receive the shares.

Share: Given (n, t) , the public parameters of vector commitment, random oracle $\mathcal{H}$, to share $k \times \ell$ secrets $\{f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)}\}_{j=1}^k$, the dealer D acts as below:

1. Sample $k + 1$ uniformly random polynomials $f^{(1)}(x), \dots, f^{(k)}(x)$ and $r(x)$ of degree $t + \ell - 1$ with coefficients in a field $\mathbb{Z}_q$ (or ring $\mathbb{Z}_N$), subject to

$$f^{(j)}(-(\ell - 1)) = f_{-(\ell-1)}^{(j)}, \dots, f^{(j)}(0) = f_0^{(j)}, \quad \text{for } j = 1, \dots, k.$$

2. For $i = 1, \dots, n$ and $j = 1, \dots, k$: compute $f_i^{(j)} := f^{(j)}(i)$, and $r_i := r(i)$.
3. For $i = 1, \dots, n$: sample $\gamma_i \leftarrow \mathbb{Z}_q$ and set $c_i = \mathcal{C}((f_i^{(1)}, \dots, f_i^{(k)}, r_i), \gamma_i)$.
4. Compute $d_1, \dots, d_k \leftarrow \mathcal{RO}(c_1, \dots, c_n)$, where $\mathcal{RO}$ is a random oracle.
5. Set $z(x) = r(x) + \sum_{j=1}^k d_j f^{(j)}(x)$ and $\pi_{share} := (c_1, \dots, c_n, z(x))$.
6. Send shares $(\{f_i^{(j)}\}_{j=1}^k, \gamma_i)$ securely to party P_i and publish π_{share} .

Verify: Given $\pi_{share} := (c_1, \dots, c_n, z(x))$, and the individual shares:

1. Each P_i checks $\deg(z(x)) \leq t + \ell - 1$; if so, it proceeds as follows:
 - (a) Computes $(d_1, \dots, d_k) \leftarrow \mathcal{RO}(c_1, \dots, c_n)$.
 - (b) Uses her/his shares $(\{f_i^{(j)}\}_{j=1}^k, \gamma_i)$, and checks whether

$$c_i = \mathcal{C}((f_i^{(1)}, \dots, f_i^{(k)}, z(i) - \sum_{j=1}^k d_j f_i^{(j)}), \gamma_i).$$

If the verification of P_i fails, then P_i broadcasts a complaint against D .

2. If the number of shareholders complaining against the dealer exceeds t , the dealer will be disqualified, and the **Verify** will return **false**.
3. In case a shareholder P_i raises a complaint about the verification of their part, the dealer will broadcast $\{f_i^{(j)} = f^{(j)}(i)\}_{j=1}^k$ and the randomizer γ_i (if applicable) to enable everyone to verify it using the verification equation. If the verification passes, the protocol continues as usual. However, if it fails, the dealer will be disqualified, leading to a **false** verification outcome. Since the disqualification decision is solely based on the information broadcasted, all honest shareholders will ultimately reach a consensus either on a set of qualified parties $Q \subseteq \{P_1, P_2, \dots, P_n\}$ or on rejecting the final verification.

Reconstruct: Given $\pi_{share} := (c_1, \dots, c_n, z(x))$, each party P_i broadcasts the share values $\{f_i^{(j)}\}_{j=1}^k$ and γ_i . A party P_i is said to be confirmed if

$$c_i = \mathcal{C}((f_i^{(1)}, \dots, f_i^{(k)}, z(i) - \sum_{j=1}^k d_j f_i^{(j)}), \gamma_i),$$

where $(d_1, \dots, d_k) \leftarrow \mathcal{RO}(c_1, \dots, c_n)$. Consider $\{f_i^{(j)}\}_{j=1}^k$ values of any $t + \ell$ confirmed parties and interpolate $f^{(1)}(x), \dots, f^{(k)}(x)$ of degree $t + \ell - 1$ that passes through those points. Finally, the output is **false** (if $t + \ell$ valid sets of shares were not obtained) or

$$f^{(j)}(-(\ell - 1)) = f_{-(\ell-1)}^{(j)}, \dots, f^{(j)}(0) = f_0^{(j)}, \quad \text{for } j = 1, \dots, k.$$

Fig. 2. (k, ℓ) -BII: A unified framework for k -batched and ℓ -packed computational verifiable secret sharing in the synchronous setting for $n \geq 2t + \ell$.

and a single masking polynomial $r(x)$ of degree $t + \ell$ with n evaluations. This totals $n(k + 1)$ polynomial evaluations, compared to the $2n(k + \ell)$ evaluations required for $k + \ell$ independent executions of Π . The process further involves generating only n commitments (one per participant, each encoding k shares) instead of $n(k + \ell)$ commitments. To construct the proof π_{share} , the dealer combines all k secret-sharing polynomials with $r(x)$ via the equation

$$z(x) = r(x) + \sum_{j=1}^k d_j f^{(j)}(x),$$

where $d_1, \dots, d_k$ are derived from a single query to the random oracle $\mathcal{RO}$. This requires fewer arithmetic operations and fewer queries to $\mathcal{RO}$ compared to $k + \ell$ separate executions of Π . Note that, in practice we can set the random oracle to return just a single random element d , and then set $d_j = d^j$ for $j = 1, \dots, k$.

During verification, each party evaluates the degree- $(t + \ell)$ polynomial $z(x)$ and verifies its consistency with the batched commitments. This step retains the computational profile of the original protocol: each party performs one polynomial evaluation (over a degree- $(t + \ell)$ polynomial), one random oracle query, and one commitment check, regardless of k . Crucially, the communication overhead is reduced: the dealer broadcasts n commitments (instead of $n(k + \ell)$) and the $t + \ell + 1$ coefficients of $z(x)$ (instead of $(k + \ell)(t + 1)$), while securely sending each participant $k + \ell$ shares, as in $k + \ell$ executions of the original protocol.

By batching proofs and leveraging ℓ -packed secret sharing, $\text{B}\Pi$ achieves nearly optimal amortized complexity; the cost per secret shrinks to $\mathcal{O}(1/k)$ for large k , while preserving the security guarantees of the base Π protocol. It is important to note that the increased polynomial degree $t + \ell$ (compared to t in Π) reflects the ℓ -packed secret-sharing paradigm but does not asymptotically dominate the efficiency gains from batching.

4 Practical Batched and Packed VSS Protocols

In this section, we first instantiate the general (k, ℓ) - $\text{B}\Pi$ framework introduced in Sec. 3 with various vector commitment schemes, and present multiple k -batched and ℓ -packed computational VSS schemes. The resulting BP-VSS schemes, denoted $\text{B}\Pi_{\text{LA}}$, $\text{B}\Pi_{\text{P}}$, $\text{B}\Pi_{\text{P}+}$, and $\text{B}\Pi_{\text{F}}$, each achieve distinct trade-offs in terms of security and efficiency. In the rest, we also introduce batched and packed version of well-known Feldman [24] and Pedersen [32] VSS schemes that both operate in the plain model. All our proposed BP-VSS schemes operate in the synchronous setting for $n \geq 2t + \ell$, where n is the number of parties, t the number of malicious parties, and ℓ the packing parameter. When $\ell = 1$ (the k -batched and non-packed case), all schemes achieve optimal resilience. On the positive side, these schemes are tunable: their performance can be further improved by setting $\ell > 1$, but at the cost of slightly reducing the tolerated number of malicious parties (on average by $\ell/2$) and sacrificing protocol robustness. In particular, combining batching and packing yields the best overall efficiency compared to using either technique alone. Thus, in practice, the parameters (k, ℓ) can be tuned to match the number of participants, the amount of secrets to be shared, and the desired robustness

level. This tunability and diversity make our schemes applicable across a broad range of use cases, each benefiting from different efficiency and security guarantees. Applications include threshold signatures, threshold decryption, e-voting, robust federated learning, distributed randomness generation, and Distributed Key Generation (DKG), among others. For instance, BII_P and BII_P^+ enable DKG protocols with information-theoretically unbiased public keys (as in [27]), while BII_{LA} and BII_F yield more efficient DKGs that achieve only computational unbiasedness. Notably, BII_{LA} is particularly well-suited for PQ secure protocols.

4.1 BII_{LA} : Batched and Packed Version of the VSS Scheme Π_{LA}

Our initial instantiation of (k, ℓ) -BII, denoted as BII_{LA} , uses a hash-based vector commitment scheme that satisfies computational hiding. This results in a batched, packed, lightweight, and plausibly post-quantum secure VSS scheme in the synchronous setting. In BII_{LA} , commitments are computed as $c_i := H(\vec{m}, \gamma_i)$, where $\vec{m}$ is the message vector and γ_i is the randomizer. We summarize the algorithms of BII_{LA} below, focusing only on the steps that differ from the general BII scheme presented in Fig. 2.

Initial: Parties agree on a random oracle $\mathcal{RO}$ and a commitment scheme $\mathcal{H}$.

Share: Given (n, t) , the public parameters, the secrets $\{f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)}\}_{j=1}^k$, the dealer D follows the steps outlined in Fig. 2, specifically, with the following deviations:

3. For $i = 1, \dots, n$: sample $\gamma_i \leftarrow \mathbb{Z}_q$ and set $c_i = \mathcal{H}((f_i^{(1)}, \dots, f_i^{(k)}, r_i), \gamma_i)$.

Verify: Given $\pi_{share} := (c_1, \dots, c_n, z(x))$, and the individual shares:

1. Each P_i checks if $\deg(z(x)) \leq t + \ell - 1$; if so, it proceeds as follows:
 - (a) Computes $(d_1, \dots, d_k) \leftarrow \mathcal{RO}(c_1, \dots, c_n)$.
 - (b) Uses her/his shares $(\{f_i^{(j)}\}_{j=1}^k, \gamma_i)$, and checks if

$$c_i = \mathcal{H}((f_i^{(1)}, \dots, f_i^{(k)}, z(i) - \sum_{j=1}^k d_j f_i^{(j)}), \gamma_i).$$

If the verification of P_i fails, then P_i broadcasts a complaint against D .

Reconstruct: Given $\pi_{share} := (c_1, \dots, c_n, z(x))$, each party P_i broadcasts the share values $\{f_i^{(j)}\}_{j=1}^k$ and γ_i . A party P_i is said to be confirmed if

$$c_i = \mathcal{H}((f_i^{(1)}, \dots, f_i^{(k)}, z(i) - \sum_{j=1}^k d_j f_i^{(j)}), \gamma_i),$$

where $(d_1, \dots, d_k) \leftarrow \mathcal{RO}(c_1, \dots, c_n)$. Consider $\{f_i^{(j)}\}_{j=1}^k$ values of any $t + \ell$ confirmed parties and interpolate $f^{(1)}(x), \dots, f^{(k)}(x)$ of degree $t + \ell - 1$ that passes through those points. Finally, the output is **false** (if $t + \ell$ valid set of shares were not obtained) or

$$f^{(j)}(-(\ell - 1)) = f_{-(\ell-1)}^{(j)}, \dots, f^{(j)}(0) = f_0^{(j)}, \quad \text{for } j = 1, \dots, k.$$

Security. Based on Theorem 1, we can argue that BII_{LA} satisfies the notions Completeness, Verifiability, and Simulatability (as defined in Definition 1) against a computationally bounded adversary.

4.2 BII_P: Batched and Packed Version of the VSS scheme II_P

Our second instantiation of (k, ℓ) -BII, denoted as BII_P, uses the generalized and vector variant of the Pedersen commitment scheme [11, 32] that is perfectly hiding and additively homomorphic. This instantiation results in a DL-based batched and packed VSS scheme that can achieve IT-security in terms of privacy. Namely, a computationally unbounded adversary who controls up to t parties, learns nothing about the secrets and the shares of other parties. In BII_P, given the random group generators $(g_1, g_2, \dots, g_k, g_{k+1})$, the commitments are computed as $c_i := g_1^{m_1} g_2^{m_2} \dots g_k^{m_k} g_{k+1}^\gamma$, where $\vec{m} := (m_1, \dots, m_k)$ is the message vector and γ is the randomizer. We summarize the algorithms of BII_P below, which instantiates the general framework (given in Fig. 2).

Initial: Parties agree on $\mathcal{RO}$ and the group generators $(g_1, \dots, g_{k+1}, g_{k+2})$.

Share: Given (n, t) , the public parameters, the secrets $\{f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)}\}_{j=1}^k$, the dealer D follows the steps outlined in Fig. 2, specifically, with the following deviations:

3. For $i = 1, \dots, n$: sample $\gamma_i \leftarrow \mathbb{Z}_q$ and set $c_i = g_1^{f_i^{(1)}} \dots g_k^{f_i^{(k)}} g_{k+1}^{r_i} g_{k+2}^{\gamma_i}$.

Verify: Given $\pi_{share} := (c_1, \dots, c_n, z(x))$, and the individual shares:

1. Each P_i checks if $\deg(z(x)) \leq t + \ell - 1$; if so, it proceeds as follows:
 - (a) Computes $(d_1, \dots, d_k) \leftarrow \mathcal{RO}(c_1, \dots, c_n)$.
 - (b) Uses her/his shares $(\{f_i^{(j)}\}_{j=1}^k, \gamma_i)$, and checks if

$$c_i = g_1^{f_i^{(1)}} \dots g_k^{f_i^{(k)}} g_{k+1}^{z(i) - \sum_{j=1}^k d_j f_i^{(j)}} g_{k+2}^{\gamma_i}.$$

If the verification of P_i fails, then P_i broadcasts a complaint against D .

Reconstruct: Given $\pi_{share} := (c_1, \dots, c_n, z(x))$, each party P_i broadcasts the share values $\{f_i^{(j)}\}_{j=1}^k$ and γ_i . A party P_i is said to be confirmed if

$$c_i = g_1^{f_i^{(1)}} \dots g_k^{f_i^{(k)}} g_{k+1}^{z(i) - \sum_{j=1}^k d_j f_i^{(j)}} g_{k+2}^{\gamma_i},$$

where $(d_1, \dots, d_k) \leftarrow \mathcal{RO}(c_1, \dots, c_n)$. Consider $\{f_i^{(j)}\}_{j=1}^k$ values of any $t + \ell$ confirmed parties and interpolate $f^{(1)}(x), \dots, f^{(k)}(x)$ of degree $t + \ell - 1$ that passes through those points. Finally, the output is **false** (if $t + \ell$ valid set of shares were not obtained) or

$$f^{(j)}(-(\ell - 1)) = f_{-(\ell-1)}^{(j)}, \dots, f^{(j)}(0) = f_0^{(j)}, \quad \text{for } j = 1, \dots, k.$$

Security. By Theorem 1, we can argue that BII_P satisfies the notions Completeness, Verifiability, and perfect Simulatability (as defined in Definition 1). Notably, in this case, the transcript simulation is perfect, meaning even a computationally unbounded adversary, capable of corrupting up to t parties, cannot extract any information about the secrets.

4.3 An Optimized Vector Commitment for $\text{B}\Pi_{\text{P}}$ in $\text{B}\Pi_{\text{P}}+$

Similar to the well-known Pedersen scheme [32] and the original Π_{P} protocol [4], the BP-VSS scheme $\text{B}\Pi_{\text{P}}$ achieves perfect simulatability. This critical property enhances its applicability in various scenarios, such as generating public keys with a uniformly random distribution [27], or random number generation. However, $\text{B}\Pi_{\text{P}}$'s reliance on the generalized vector variant of the Pedersen commitment introduces efficiency trade-offs. Specifically, the sharing phase requires $k(n + 2)$ exponentiations, and the verification demands $k + 2$ exponentiations.

In this section, we first propose a novel vector commitment scheme that is perfectly hiding but lacks additive homomorphism, but it allows re-randomization. By combining hash functions with Pedersen commitments, our construction achieves greater efficiency than the generalized Pedersen variant [11, 32], albeit at the cost of homomorphic properties. Building on this foundation, we instantiate the framework from Fig. 2 using our hash- and DL-based vector commitment, yielding an optimized variant of $\text{B}\Pi_{\text{P}}$, called $\text{B}\Pi_{\text{P}}+$.

An Efficient Perfectly Hiding Non-Homomorphic Commitment. Let, $\text{VC} = (\text{Setup}, \mathcal{C}, \text{Verify})$ be an efficient (basic) vector commitment scheme over the message space $\mathbb{Z}_q$ that is defined as follows:

1. $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: On input the security parameter 1^λ , this algorithm samples a hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ and two distinct and random group generators (g_1, g_2) and outputs public parameters $(\mathcal{H}, g_1, g_2)$. All subsequent algorithms take $(\mathcal{H}, g_1, g_2)$ as input, but we omit it for notational clarity.
2. $\mathcal{C}(m_1, \dots, m_k; r) \rightarrow \text{c}$: On input a vector $(m_1, \dots, m_k) \in \mathbb{Z}_q^k$ and a randomizer $r \in \mathbb{Z}_q$, this algorithm outputs a commitment $\text{c} = g_1^{\mathcal{H}(m_1, m_2, \dots, m_k)} g_2^r$.
3. $\text{Verify}(\text{c}, m_1, \dots, m_k, r) \rightarrow \{0, 1\}$: Given c , a vector $(m_1, \dots, m_k)$, and a value r , this algorithm outputs 1 (accept) if $\text{c} = g_1^{\mathcal{H}(m_1, m_2, \dots, m_k)} g_2^r$; and 0 (reject) otherwise.

Theorem 2 (Efficient Perfectly Hiding Vector Commitment). *Let $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ be a random oracle mapping inputs to elements in $\mathbb{Z}_q$. The vector commitment $\text{VC} = (\text{Setup}, \mathcal{C}, \text{Verify})$ described above, satisfies completeness, perfect hiding and computational binding under the DL assumption and the collision resistance of $\mathcal{H}$.*

Proof. The proof is given in App. C.2.

Efficiency. The proposed vector commitment achieves superior efficiency to the generalized vector Pedersen commitment. Specifically, both the sharing and verification phases require only 2 exponentiations and 1 hash operation.

$\text{B}\Pi_{\text{P}}+$: An Optimized Batched and Packed Version of Π_{P} . By instantiating (k, ℓ) - $\text{B}\Pi$ with the proposed vector commitment scheme, where commitments are $c_i := g_1^{\mathcal{H}(f_i^{(1)}, \dots, f_i^{(k)}, r_i)} g_2^{\gamma_i}$ for $\gamma_i \in \mathbb{Z}_q$, we derive $\text{B}\Pi_{\text{P}}+$, a more efficient

batched and packed extension of Π_P [4]. Compared to BII_P , BII_P+ reduces exponentiations by a factor of k in both sharing and verification, at the cost of adopting a non-homomorphic commitment scheme that may limit applicability. Nevertheless, it remains suitable for practical use cases such as distributed key generation [27]. Core algorithms of BII_P+ , differing from BII_P (given in Sec. 4.2):

Initial: Parties agree on a random oracle $\mathcal{RO}$, a hash function $\mathcal{H} : 0, 1^* \rightarrow \mathbb{Z}_q$, and random generators (g_1, g_2) .

Share: Same as BII_P , except $c_i = g_1^{\mathcal{H}(f_i^{(1)}, \dots, f_i^{(k)}, r_i)} g_2^{\gamma_i}$.

Verify: Same as BII_P , except parties check $c_i = g_1^{\mathcal{H}(f_i^{(1)}, \dots, f_i^{(k)}, z(i) - \sum_{j=1}^k d_j f_i^{(j)})} g_2^{\gamma_i}$.

Reconstruct: Same as BII_P , except P_i is confirmed if $c_i = g_1^{\mathcal{H}(f_i^{(1)}, \dots, f_i^{(k)}, r_i)} g_2^{\gamma_i}$.

Security. By Theorem 1, similar to BII_P , BII_P+ satisfies Completeness, Verifiability, and perfect Simulatability.

4.4 BII_F : Batched and Packed VSS for High-Entropy Secrets

While all prior BP-VSS schemes from Subsections 4.1, 4.2, and 4.3 are direct instantiations of BII , we now propose BII_F , a novel BP-VSS scheme leveraging a generalized vector variant of Pedersen commitments [32]. Unlike previous constructions, BII_F omits the randomizer γ_i and defines commitments as $c_i := g_1^{f_i^{(1)}} \dots g_k^{f_i^{(k)}} g_{k+1}^{r_i}$. BII_F serves as a batched and packed extension of Π_F [4]. Similar to Feldman VSS [24] and Π_F , BII_F requires high-entropy secrets and achieves unpredictability, a weaker security notion compared to the simulatability guaranteed by Π_{LA} , BII_P , and BII_P+ . While this limitation might narrow its applicability, BII_F remains well-suited for distributed protocols where parties share high-entropy secrets, e.g., DKG protocols [27].

The construction of BII_F can be derived from the description of BII_P , summarized in Section 4.2, by simply removing the **highlighted** terms.

Security. We formalize the security of BII_F in the following theorem:

Theorem 3 (Batched and Packed VSS from DL). *Let $(g_1, \dots, g_{k+1})$ be $k+1$ distinct random generators of a group $\mathbb{G}$, and let $\mathcal{H}$ be a random oracle. Under the DL assumption, BII_F is a secure batched (and packed) VSS protocol in the synchronous setting for $n \geq 2t + \ell$. Specifically, it satisfies the following properties (as defined in Definition 1): (i) Completeness: An honest dealer will convince honest shareholders, and any subset of $t + \ell$ honest shareholders can reconstruct the secrets. (ii) Verifiability: The reconstruction protocol outputs a unique set of secrets distributed by the dealer for any qualified set of shareholders (i.e., of size at least $t + \ell$). (iii) Unpredictability: Any set of up to t shareholders cannot predict any secret with non-negligible advantage.*

Proof. The proof is given in App. C.3.

4.5 Batched and Packed Variants of Pedersen and Feldman VSS

The non-batched and non-packed VSS schemes (Π_P, Π_F) were proposed as alternatives to the classic Pedersen [32] and Feldman [24] VSS schemes. In this section, we also introduce batched and packed versions of both Pedersen and Feldman VSS schemes that outperform multiple sequential executions of their base protocols. Like their non-batched counterparts, both proposed protocols operate in the plain model.

BP-Pedersen: A Batched and Packed Variant of Pedersen VSS. We present BP-Pedersen, a batched and packed adaptation of Pedersen VSS supporting both batched and packed verifiable secret sharing.

Share: Given (n, t) , $k+1$ independent generators $g_1, \dots, g_k, g_{k+1}$, and $k \times \ell$ secrets $\{f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)}\}_{j=1}^k$, the dealer D proceeds:

1. Generate k uniformly random degree $t + \ell - 1$ polynomials:

$$f^{(j)}(x) := f_0^{(j)} + a_1^{(j)}x + \dots + a_{t+\ell-1}^{(j)}x^{t+\ell-1}$$

for $j = 1, \dots, k$, subject to $f^{(j)}(-(\ell-1)) = f_{-(\ell-1)}^{(j)}, \dots, f^{(j)}(0) = f_0^{(j)}$.

2. Generate random polynomial: $r(x) = r_0 + b_1x + \dots + b_{t+\ell-1}x^{t+\ell-1}$ with coefficients in $\mathbb{Z}_q$.
3. Compute $f_i^{(j)} := f^{(j)}(i)$ and $r_i := r(i)$ for $i = 1, \dots, n, j = 1, \dots, k$
4. Construct commitments: $c_0 = g_{k+1}^{r_0} \prod_{j=1}^k g_j^{f_0^{(j)}}$ and $c_v = g_{k+1}^{b_v} \prod_{j=1}^k g_j^{a_v^{(j)}}$ for $v = 1, \dots, t + \ell - 1$. Broadcast $\pi_{\text{share}} := (c_0, c_1, \dots, c_t)$ and send $\{r_i, f_i^{(j)}\}_{j=1}^k$ to each P_i .

Verify. Verification requires each P_i to check: $g_{k+1}^{r_i} \prod_{j=1}^k g_j^{f_i^{(j)}} = \prod_{v=0}^{t+\ell-1} c_v^{i_v}$.

Efficiency and Security. Compared to k sequential Pedersen executions, BP-Pedersen reduces: (1) Broadcast communication by a factor k , (2) Verification cost from $\mathcal{O}(kn)$ to $\mathcal{O}(k+n)$ exponentiations. Assuming an unknown link between the generators $(g_1, \dots, g_k, g_{k+1})$, BP-Pedersen's security follows analogously to Pedersen's VSS [32], achieving perfect simulatability and computational verifiability reduced to the DL problem.

BP-Feldman: A Batched and Packed Variant of Feldman VSS. Using similar batching techniques, we can also construct BP-Feldman VSS that requires high-entropy secrets. The construction of BP-Feldman can be derived from the description of BP-Pedersen, summarized above, by simply removing the highlighted terms. In terms of efficiency, compared to k sequential Feldman executions, BP-Feldman reduces: (1) Broadcast communication by factor k , (2) Verification cost from $\mathcal{O}(kn)$ to $\mathcal{O}(k+n)$ exponentiations. In terms of security, its security follows analogously to Feldman VSS [24], reducible to the DL problem.

Notably, while standard batching techniques [5] could theoretically batch the verification of multiple Feldman/Pedersen VSS executions, our analysis shows

these approaches would be k -times less efficient than our proposed variants in both proof size and verification. This efficiency gap stems from a key architectural difference: our construction assumes a *single dealer* generating one proof for all $k \times n$ shares. Traditional verification batching handles proofs from k *different dealers*, only aggregating verification for $k \times n$ shares. This shows that in applications where all shares are generated by a single dealer, by shifting batching to the proof-generation phase rather than just verification, we can construct even more efficient protocols.

5 An Efficient Batched and Packed PVSS

In all proposed batched-and-packed VSS schemes in Section 4, share correctness can only be verified by shareholders. In this section, we propose a Batched-and-Packed Publicly Verifiable Secret Sharing (BP-PVSS) scheme based on the BII_F BP-VSS construction. The new BP-PVSS protocol BII_S can be viewed as a batched variant of the PVSS scheme Π_S proposed in [4, 14].

To achieve this, we first develop a batched version of the sigma protocol π_{PDL} for the Polynomial Discrete Logarithm (PDL) problem (Definition 6), recently introduced by Baghery [4] and employed in Π_S PVSS.

5.1 Batched Version of Baghery's π_{PDL} Protocol for PDL

This section presents a batched variant of the π_{PDL} protocol from [4, Sec. 5]. The original protocol extends Schnorr's identification scheme to polynomial domains. While π_{PDL} enables knowledge proofs for k degree- t polynomials, this requires k protocol executions - an inefficient approach. To overcome this limitation, we propose a batched π_{PDL} protocol allowing a prover to demonstrate witness knowledge for the relation R_{PDL}^{Mul} defined as:

$$R_{PDL}^{\text{Mul}} = \left\{ \left(g, \{\alpha_i, \{F_i^{(j)}\}_{j=1}^k\}_{i=1}^n \right), \{f^{(j)}(x)\}_{j=1}^k \mid \begin{array}{l} F_1^{(j)} = g^{f^{(j)}(\alpha_1)}, \dots, F_n^{(j)} = \\ g^{f^{(j)}(\alpha_n)}, \quad \text{for } j = 1, \dots, k \end{array} \right\},$$

where $\{f^{(1)}(x), \dots, f^{(k)}(x)\} \in \mathbb{Z}_q[x]_t$ are witness polynomials of degree at most $t \leq n-1$ with coefficients in $\mathbb{Z}_q$, and $\alpha_1, \dots, \alpha_n$ are n distinct elements from $\mathbb{Z}_q$.

Fig. 3 describes our proposed batched version of π_{PDL} protocol, which enables a prover to generate a NIZK proof of knowledge for the relation R_{PDL}^{Mul} .

Theorem 4 (A NIZK Proof of Knowledge for R_{PDL}^{Mul}). *Let g be the generator of $\mathbb{G}$, $\{F_1^{(j)}, \dots, F_n^{(j)}\}_{j=1}^k \in \mathbb{G}^{n \times k}$, $\{\alpha_i\}_{i=1}^n$ be n distinct elements from $\mathbb{Z}_q$, and t be the (maximum) degree of witness polynomials $f^{(1)}(x), \dots, f^{(k)}(x)$. Assuming PDL is hard, for $0 \leq t < n$, the protocol π_{BPDL} (given in Fig. 3) is a NIZK PoK for R_{PDL}^{Mul} in the RO model.*

Proof. The proof is given in App. C.4.

Prover: Given the statement $(g, \alpha_1, \dots, \alpha_n, F_1^{(j)}, \dots, F_n^{(j)})_{j=1}^k$ and the witness polynomials $\{f^{(j)}(x)\}_{j=1}^k$, proceed as follows and output a NIZK proof π .

1. Sample a degree- t polynomial $r(x) \in \mathbb{Z}_q[x]_t$; Set $\{\Gamma_i = g^{r(\alpha_i)}\}_{i=1}^n$.
2. Set $d \leftarrow \mathcal{H}(\{F_1^{(j)}, \dots, F_n^{(j)}\}_{j=1}^k, \Gamma_1, \dots, \Gamma_n)$, where $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ a RO.
3. Set $z(x) = r(x) + \sum_{j=1}^k d^j f^{(j)}(x) \pmod{q}$;
4. Return $\pi := (\Gamma_1, \dots, \Gamma_n, z(x))$ or $\pi := (d, z(x))$

Verifier: Given the statement $(g, \alpha_1, \dots, \alpha_n, F_1^{(j)}, \dots, F_n^{(j)})_{j=1}^k$ and $\pi := (\Gamma_1, \dots, \Gamma_n, z(x))$, the verifier first checks if $z(x)$ is a degree- t polynomial. If so, then sets $d \leftarrow \mathcal{H}(\{F_1^{(j)}, \dots, F_n^{(j)}\}_{j=1}^k, \Gamma_1, \dots, \Gamma_n)$ and checks if: $g^{z(\alpha_i)} = \Gamma_i \prod_{j=1}^k (F_i^{(j)})^{d^j}$ for $i = 1, \dots, n$, and outputs **true** or **false**. Note that to make the communication shorter, alternatively, the prover could publish $\pi := (d, z(x))$, and then the verifier would need to check if $z(x)$ is a degree- t polynomial and $d = \mathcal{H}(\{F_1^{(j)}, \dots, F_n^{(j)}\}_{j=1}^k, \frac{g^{z(\alpha_1)}}{\prod_{j=1}^k (F_1^{(j)})^{d^j}}, \dots, \frac{g^{z(\alpha_n)}}{\prod_{j=1}^k (F_n^{(j)})^{d^j}})$.

Fig. 3. π_{BPDF} : Our proposed batched version of π_{PDF} .

5.2 BII_S: An Efficient Batched and Packed PVSS

Given the NIZK argument π_{BPDF} , described in Fig. 3, next we construct BII_S, that can be considered as a batched version of the PVSS scheme II_S. In a typical PVSS scheme, the dealer encrypts each share using the public key of the receiving party and also attaches a non-interactive publicly verifiable ZK proof to prove the correctness of the encrypted shares. In our case, we use our proposed batched NIZK argument to give a batched proof for k parallel and packed executions of the protocol. Intuitively, the proposed BP-PVSS scheme uses the homomorphic property of the encryption to perform BII-like verification in the exponent.

Construction. Given a group generator g of group $\mathbb{G}$, party P_i samples $s_i \leftarrow \mathbb{Z}_q$ and registers the public key $\text{pk}_i = g^{s_i}$. We also assume that each party has given a valid proof of knowledge of their secret key. These public keys are known to all parties and can be reused. Note that similar to k executions of ℓ -packed II_S, at the end of the reconstruction phase, each party P_i will obtain k distinct shares $(g^{f^{(1)}(i)}, \dots, g^{f^{(k)}(i)})$ and later in the reconstruction phase, any qualified set of parties will be able to reconstruct all the $k \times \ell$ secrets $\{g^{f^{(j)}_{-(\ell-1)}}, \dots, g^{f^{(j)}_0}\}_{j=1}^k$. More precisely, in the reconstruction phase, each party broadcasts the decryption of their k shares. To maintain security, this should be accompanied with a proof of valid decryption. This we achieve by using the NIZK proof of knowledge proposed by Chaum, Evertse, and Graaf [20], which is summarized in Fig. 8.

We summarize the algorithms of BII_S below. In the sharing phase, we focus only on the steps deviating from the general BII scheme.

Initial: Parties agree on a random oracle $\mathcal{RO}$, group generator g , register public keys $\text{pk}_i = g^{s_i} i = 1^n$, and store secret keys $s_i i = 1^n$.

Share: Given (n, t) , public keys $\{\text{pk}_i\}_{i=1}^n$, and secrets $\{f^{(j)}_{-(\ell-1)}, \dots, f^{(j)}_0\}_{j=1}^k$, the dealer D proceeds:

1. Sample k random polynomials $f^{(1)}(x), \dots, f^{(k)}(x)$ of degree $t+\ell-1$ with coefficients in $\mathbb{Z}_q$, satisfying $f^{(j)}(-(\ell-1)) = f_{-(\ell-1)}^{(j)}, \dots, f^{(j)}(0) = f_0^{(j)}$.
 2. For $i = 1, \dots, n$ and $j = 1, \dots, k$: compute encrypted shares $y_i^{(j)} = \text{pk}_i^{f^{(j)}(i)}$.
 3. Run a modified π_{BPDL} (Fig. 3) to generate a batch proof π_{share} :
 - Sample random polynomial $r(x)$ of degree $t+\ell-1$ and set $\Gamma_i = \text{pk}_i^{r(i)}$.
 - Compute $d \leftarrow \mathcal{RO}(\{y_1^{(j)}, \dots, y_n^{(j)}\}_{j=1}^k, \Gamma_1, \dots, \Gamma_n)$ and set $z(x) = r(x) + \sum_{j=1}^k d^j f^{(j)}(x)$.
 - Set $\pi_{share} := (d, z(x))$.
 4. Publish encrypted shares $\{y_1^{(j)}, \dots, y_n^{(j)}\}_{j=1}^k$ and π_{share} .
- Verify:** Given $\{\text{pk}_i\}_{i=1}^n$, ciphertexts $\{y_1^{(j)}, \dots, y_n^{(j)}\}_{j=1}^k$, and $\pi_{share} = (d, z(x))$, a verifier checks that $z(x)$ is degree $\leq t+\ell-1$ and

$$d = \mathcal{H}(\{y_1^{(j)}, \dots, y_n^{(j)}\}_{j=1}^k, \frac{\text{pk}_1^{z(1)}}{\prod_{j=1}^k (y_1^{(j)})^{d^j}}, \dots, \frac{\text{pk}_n^{z(n)}}{\prod_{j=1}^k (y_n^{(j)})^{d^j}}),$$

returning **true** or **false**.

Reconstruction: Parties proceed as follows:

1. Each party decrypts shares $\{F_i^{(j)} := g^{f^{(j)}(i)}\}_{j=1}^k$ via $F_i^{(j)} = (y_i^{(j)})^{1/s_i}$ and publishes them with a NIZK proof (Fig. 8 [20]) that $(\text{pk}_i = g^{s_i}) \wedge_{j=1}^k (y_i^{(j)} = (F_i^{(j)})^{s_i})$.
2. For each $j \in [k]$, given $t+\ell$ valid $F_i^{(j)}$, the secrets $g^{f^{(j)}(-(\ell-1))}, \dots, g^{f^{(j)}(0)}$ are recovered via Lagrange interpolation:

$$\prod_{i=1}^{t+\ell} (F_i^{(j)})^{\lambda_i} = g^{f^{(j)}(0)} = g^{f_0^{(j)}},$$

$$\text{with } \lambda_i^{(j)} = \prod_{m=1, m \neq i}^{t+\ell} \frac{m}{m-i}.$$

An analysis of the efficiency of BII_S and its non-batched variant is provided in Table 2. We prove the security of new scheme according to the definition used in [4, 34]. The security of the proposed scheme is inherited from the original constructions in [4, 14], combined with the batched and packed NIZK argument introduced in Fig. 3, whose security guarantees are formally established in this work. For the sake of completeness, we rigorously restate the security arguments in the theorem below.

Theorem 5 (Security of BP-PVSS Scheme BII_S). *Under the PDL and Decisional Diffie-Hellman (DDH) assumptions, the VSS scheme BII_S described above, is a secure PVSS scheme against an static adversary in the random oracle model. That is, (i) the Reconstruct protocol results in the secret distributed by the dealer for any qualified set of shareholders, (ii) any non-qualified set of shareholders is unable to recover any (partial) information on the secret.*

Proof. The proof is given in App. C.5.

6 Implementation Results

We analysed the asymptotic costs of our BP-VSS schemes - BII_{LA} , BII_{P} , $\text{BII}_{\text{P}+}$, BII_{F} , BP-Feldman, BP-Pedersen - and the BP-PVSS scheme BII_{S} , comparing them with multiple executions of their non-batched versions from [4, 24, 32]. Detailed communication and computation costs are in Tables 1 and 2.

We report implementation results under various parameters based on a prototype Rust implementation³. Experiments ran on a MacBook Pro with M4 Pro CPU and 24GB RAM, applying identical optimizations across all protocols. Non-batched variants were also benchmarked for reference. Each data point is averaged over 100 runs. We focus on $\ell = 1$ (non-packed); for $\ell > 1$, performance should scale proportionally, making the packed-and-batched schemes even faster.

Performance of New VSS Schemes. Our implementation results of Shamir [36] and VSS schemes (Π_{F} , Π_{P} , Π_{LA}) [4], as well as the new BP-VSS schemes (BII_{F} , BII_{P} , $\text{BII}_{\text{P}+}$, BII_{LA}), are summarized in the plots presented in Fig. 4, 5, and 6. Our evaluation covers key metrics like sharing time, verification time, and communication cost. All results are measured in the optimistic (“happy path”) setting, where no one complains. This allows us to isolate the efficiency improvements offered by batching while preserving comparability across schemes.

Sharing Time. The plots in Fig. 4 reports the sharing time of the proposed BP-VSS schemes. We compare these results against their basic counterparts from [4] as well as the original (non-verifiable) Shamir secret sharing [36]. This comparison allows us to quantify the additional cost introduced by verifiability. As can be seen, both BII_{LA} and $\text{BII}_{\text{P}+}$ exhibit performance very close to Shamir scheme, particularly when sharing a large number of secrets. For instance, when sharing more than 100 secrets, the overhead of BII_{LA} is below 3%, while for $\text{BII}_{\text{P}+}$ it remains around 8%. These results demonstrate that BII_{LA} enables verifiability in Shamir secret sharing in post-quantum and large-scale settings at almost no additional cost. This efficiency suggests that many existing protocols relying on Shamir secret sharing can be revisited and improved by substituting with BII_{LA} to obtain verifiability essentially *for free*.

Verification Time. The plots in Fig. 5 evaluate the verification time of each scheme. We observe that BII_{LA} again achieves the best performance: each party can verify 100 received shares in roughly 3 microseconds. Similarly, the $\text{BII}_{\text{P}+}$ scheme, which additionally provides information-theoretic privacy, requires around 8 microseconds. Despite this increase, the verification remains extremely fast in practice and well within the range required for large-scale deployments.

Broadcast Communication. The plots in Fig. 5 focuses on broadcast communication overhead for different VSS schemes, measured for $n \in \{64, 128, 256\}$ and various values of k ranging from 1 to 100. Here, the benefits of batching become

³ Our open-source code will be publicly available on the COSIC Git page at <https://github.com/KULeuven-COSIC>.

clear: broadcast communication in batched protocols decreases significantly, by a factor of k . This reduction translates directly into efficiency gains for higher-level protocols that build on top of VSS, as communication often represents a major performance bottleneck in distributed systems.

Performance of the Batched Pedersen VSS Scheme. We also implemented the Feldman and Pedersen VSS protocols [24, 32], as well as our proposed batched and packed variants of them. For $(n, t) = (256, 127)$ and $k = 10$, the sharing phase highlights the efficiency gap: Shamir requires 2.95 ms, while Pedersen incurs a 395% overhead at 14.61 ms. Our batched Pedersen variant (from Sec. 4.5) reduces this to 9.62 ms (226% overhead), and the new BP-VSS scheme BII_P+ (from Sec. 4.3) achieves 4.89 ms (65% overhead). Importantly, for larger values of k , this overhead decreases significantly in our protocols, further improving their scalability. The verification phase further amplifies these gains: Pedersen requires 5.33 ms for verification, while batched Pedersen reduces this to 0.61 ms. By contrast, BII_P+ brings the verification time down dramatically to 0.0068 ms. These results clearly show that our batched variants can significantly outperform the original Pedersen scheme, with BII_P+ offering near-Shamir efficiency in sharing and orders-of-magnitude improvements in verification.

Performance of the new BP-PVSS Scheme BII_S . We further benchmarked our new BP-PVSS scheme BII_S and compared it with its original variant from [4]. For completeness, in evaluating the sharing time and communication size, we also included a passive-secure baseline where the dealer applies Shamir secret sharing and sends encrypted shares without any proof to the parties, corresponding to the minimal possible cost even when the dealer is fully trusted. Fig. 7 summarizes the benchmark results for our new BP-PVSS scheme compared to its original variant and the passive baseline. The results show that our proposed batched design achieves substantial efficiency improvements as the number of shared secrets increases. Importantly, the overhead of our scheme compared to the passive baseline quickly becomes negligible. While a single sharing incurs roughly 88% extra cost due to proof generation, this overhead decreases sharply as the number of secrets grows –dropping to only 1.3% when $k = 250$. Beyond $k \geq 100$, the additional cost remains below 2%, meaning the performance of BII_S becomes indistinguishable from the passive baseline despite offering full public verifiability. In terms of communication, the proposed batched construction also achieves near-optimal efficiency. The original non-batched scheme incurs a constant 50% overhead compared to the baseline, as it must transmit $(t + 2)k$ proof elements. In contrast, our new batched design aggregates all proofs into a single constant-size batch of $(t + 2)$ elements, independent of k . For example, with $(n, t) = (256, 127)$ and $k = 250$, the total communication decreases from 3000 KB in the original scheme to only 2004 KB in BII_S , almost matching the 2000 KB of the passive baseline. Overall, the additional communication cost remains below 1% for all tested parameters and falls to just 0.2% for large batches, confirming that BII_S achieves both computational and communication efficiency essentially equivalent to the baseline while maintaining full public verifiability.

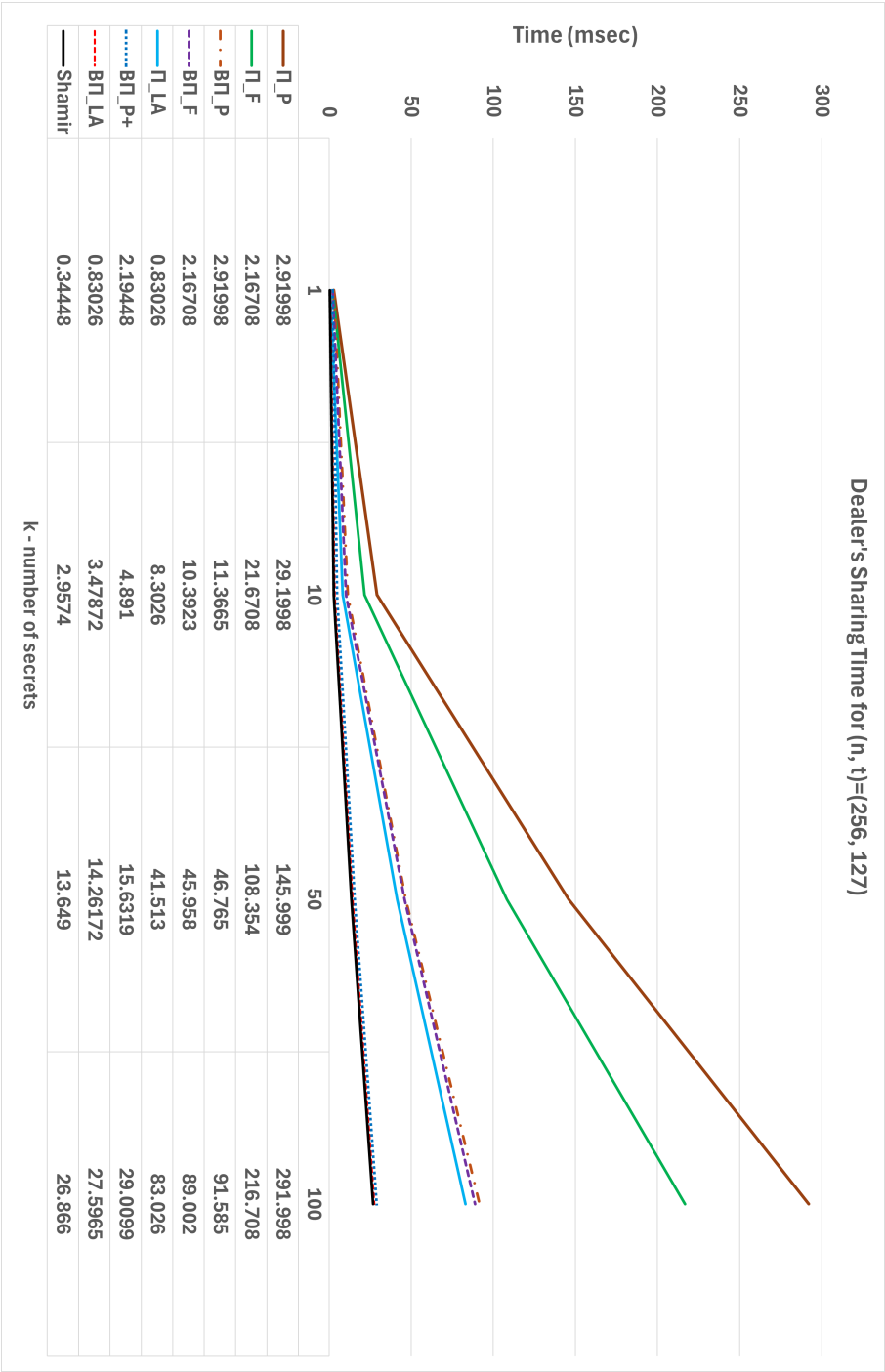


Fig. 4. Sharing time in Π_F , Π_P , Π_{LA} , $B\Pi_F$, $B\Pi_P$, $B\Pi_{P+}$, $B\Pi_{LA}$ and (baseline) Shamir.

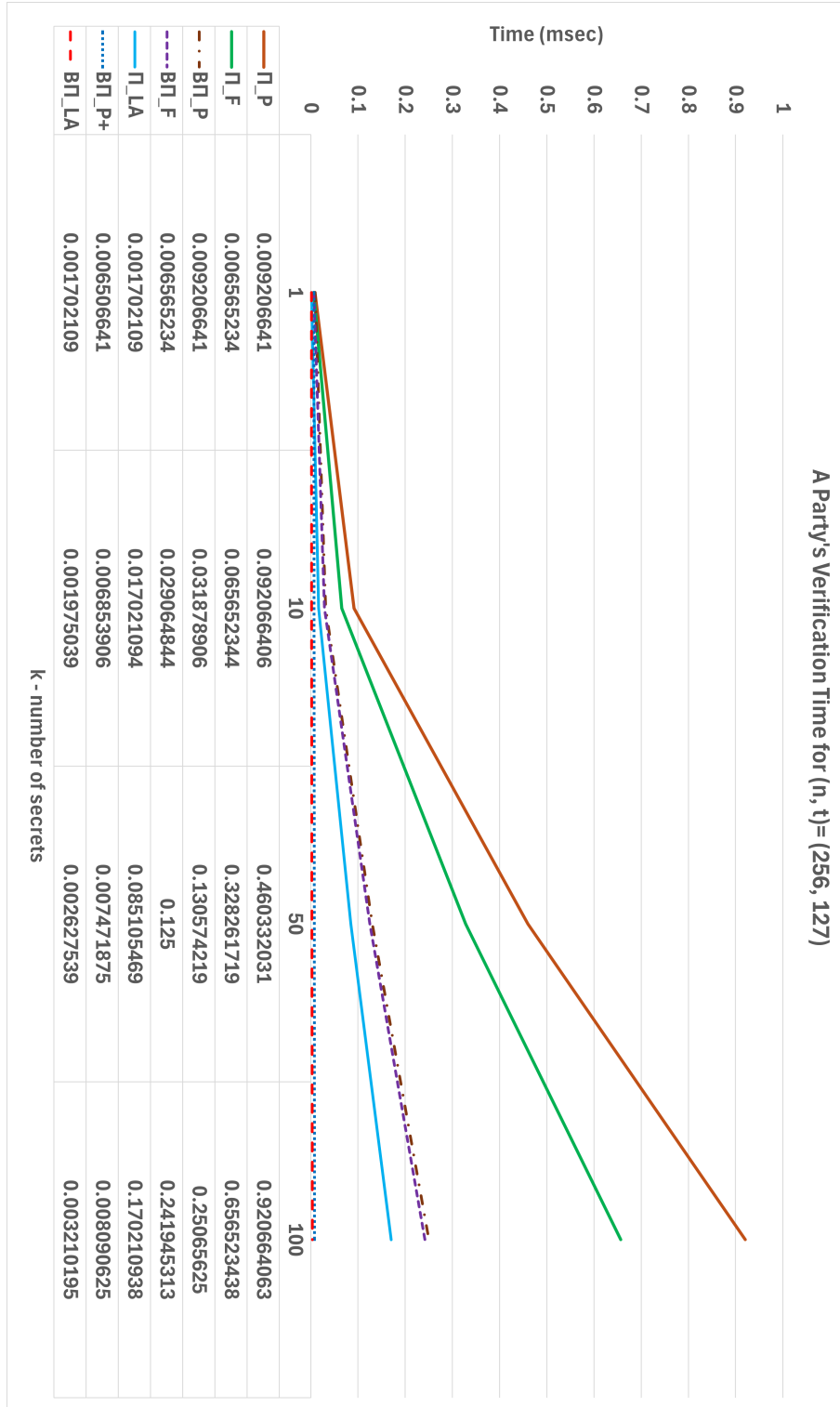


Fig. 5. Verification time in VSS schemes Π_F , Π_P , Π_{LA} , $B\Pi_F$, $B\Pi_P$, $B\Pi_{P+}$, $B\Pi_{LA}$.

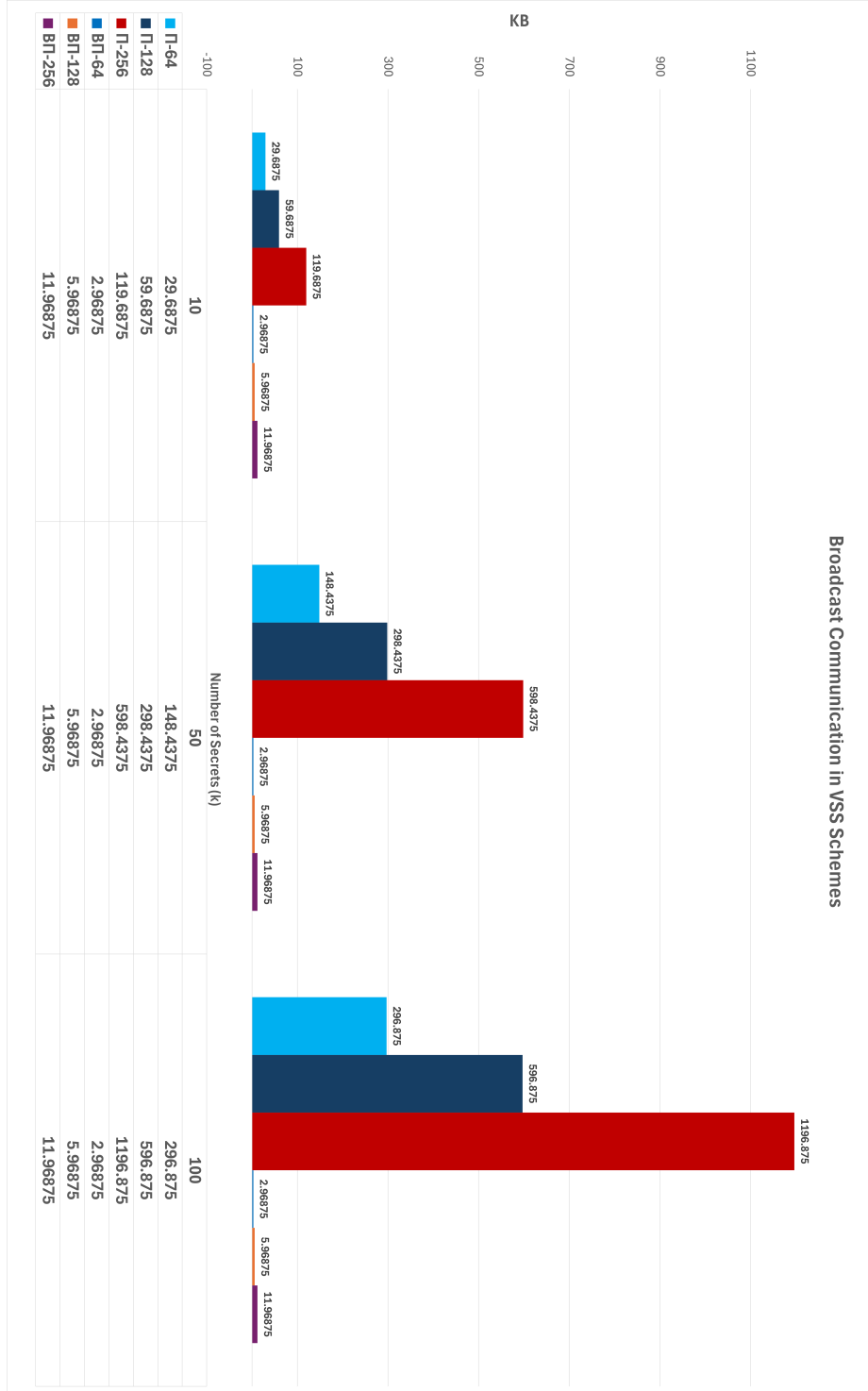


Fig. 6. Broadcast communication in schemes built by Π and Π for $n = 64, 128, 256$.

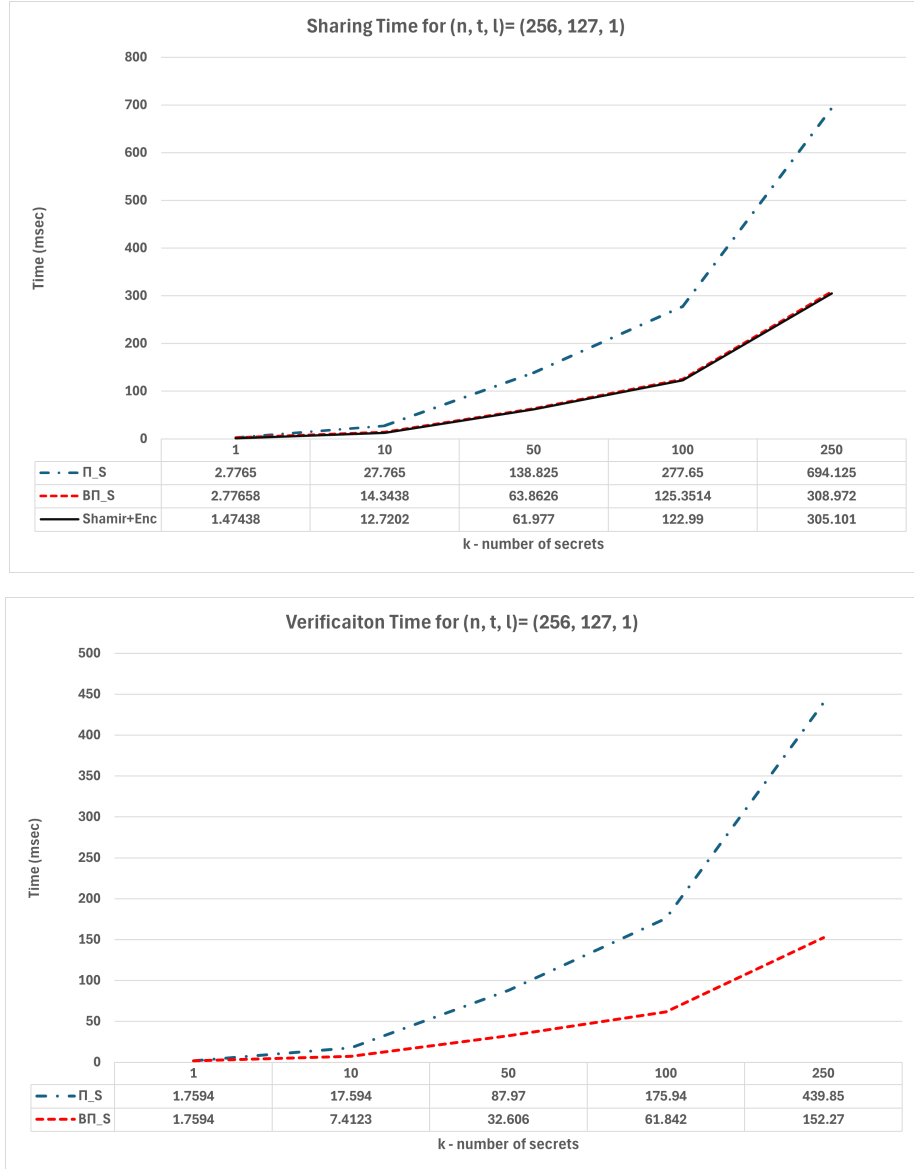


Fig. 7. Benchmark results comparing the performance of the proposed batched packed PVSS scheme ($B\Pi_S$) with its original variant Π_S and the passive baseline for $(n, t, l) = (256, 127, 1)$ and varying numbers of secrets k .

7 ALBATROSS+: Boosting ALBATROSS with Batching

7.1 ALBATROSS Randomness Generation Protocol

ALBATROSS [14] is a multiparty randomness generation protocol that combines PVSS with extraction techniques to efficiently produce uniformly random values. At its core, it uses a packed variant of the Π_S PVSS scheme [4, 14], allowing each party to secret-share ℓ secrets via a single polynomial. This reduces cost by encoding multiple secrets into one sharing instance. The protocol then applies linear t -resilient functions (via FFT) to extract ℓ^2 random values, even with up to t corruptions. Thus, ALBATROSS achieves $O(1)$ amortized exponentiations per party per value under the DDH assumption, outperforming prior works such as SCRAPE [13]. For a comparative study of randomness protocols, see [31].

7.2 Limitations of Packed Secret Sharing and Protocol Repetition

Packed Shamir secret sharing requires $n \geq 2t + \ell$ for privacy and correctness, a constraint inherited by ALBATROSS. For fixed n , this bounds $\ell \leq n - 2t$, so increasing ℓ demands larger n or smaller t . Reducing t weakens robustness and fails for large ℓ . When n is small, ℓ is severely limited, forcing repeated executions to scale randomness. For example, with $n = 7$, $t = 1$, one can share at most $\ell = 5$ secrets, yielding $\ell^2 = 25$ random values. To obtain $L = 1000$ values requires $k = \frac{1000}{25} = 40$ runs, greatly increasing overhead. More precisely, the amortized cost rises from $O(1)$ to $O(k)$, where $k = \frac{L}{(n-2t)^2}$.

7.3 ALBATROSS with Batching: Eliminating Repetition Overheads

To enhance efficiency, we propose ALBATROSS+, which integrates our batched and packed PVSS (Section 5). By batching multiple PVSS instances into one execution, ALBATROSS+ shares secret vectors across different polynomials (each satisfying $n \geq 2t + \ell$) while reusing common algebraic structures. This enables aggregated proofs and verifications, eliminating repetition without losing public verifiability or security. With n parties, ALBATROSS+ generates $(\ell k)^2$ random values in a single run, matching repeated executions but with sublinear cost growth. Thus, the amortized per-value cost remains $O(1)$, even for small n , making ALBATROSS+ highly scalable for real-world randomness generation.

The ALBATROSS+ protocol replaces the packed Π_S with Π_S and, like the original, proceeds in four phases among n parties with at most $t < \frac{n-\ell}{2}$ corruptions. Setting $\ell = n - 2t$, parties use a public ledger to post information, assuming BP-PVSS initialization and registered keys $\text{pk}_{i=1}^n$. They also agree on a Vandermonde $(n - 2t) \times (n - t)$ matrix $M = M(\omega, n - 2t, n - t)$ with $\omega \in \mathbb{Z}_q^*$.

Commit Phase: For $1 \leq m \leq n$:

- Party P_m executes **Share** phase of Π_S as Dealer for $k \times \ell$ secrets, where $\ell = n - 2t$. Publishes encrypted shares $y_{m,1}^{(j)} = \text{pk}_1^{f_m^{(j)}(1)}, \dots, y_{m,n}^{(j)} =$

$\text{pk}_n^{f_m^{(j)}(n)}$ for $j = 1, \dots, k$ along with the NIZK batched proof $\pi_{\text{share}} := (d, z(x))$. The party learns the secrets $\{g^{f_{-(\ell-1),m}^{(j)}}, \dots, g^{f_{0,m}^{(j)}}\}_{j=1}^k$ and the exponents $\{f_{-(\ell-1),m}^{(j)}, \dots, f_{0,m}^{(j)}\}_{j=1}^k$.

- Reveal Phase:**
1. All parties first verify the encrypted shares via **Verify** algorithm of new BP-PVSS scheme.
 2. Once $n - t = t + \ell$ valid set of sharings are posted (set $\mathbb{C}$), each party $P_m \in \mathbb{C}$ reveals their k sharing polynomials $f_m^{(1)}(x), \dots, f_m^{(k)}(x)$.
 3. Every party now verifies that indeed $f_m^{(1)}(x), \dots, f_m^{(k)}(x)$ match committed shares by recomputing $y_{m,1}^{(j)} = \text{pk}_1^{f_m^{(j)}(1)}, \dots, y_{m,n}^{(j)} = \text{pk}_n^{f_m^{(j)}(n)}$ for $j = 1, \dots, k$.
 4. If all parties in $\mathbb{C}$ open correctly, go to *Alternative output* phase, otherwise proceed to the *Recovery* phase.

- Recovery:**
1. Let $\mathbb{C}_A \subseteq \mathbb{C}$ be parties failing to open secret polynomials.
 2. All parties execute amortized share decryption for BP-PVSSs from $\mathbb{C}_A$.
 3. Verify decrypted shares via verification of k -DLEQ proofs (given in Fig. 8).
 4. Once $n - t = t + \ell$ valid set of shares are published (set $\mathbb{Q}$), reconstruct secrets using the reconstruction algorithm of BII_S scheme.

- Output:**
1. Let T be $(n - t) \times (\ell k)$ matrix with rows indexed by the parties in $\mathbb{C}$, and where the row corresponding to P_m is $(g^{f_{-(\ell-1),m}^{(1)}}, \dots, g^{f_{0,m}^{(1)}}), \dots, (g^{f_{-(\ell-1),m}^{(k)}}, \dots, g^{f_{0,m}^{(k)}})$.
 2. Each computes the $\ell k \times \ell k$ -matrix $R = M \circ T$ by applying FFTE on each column $T^{(m)}$ of T , resulting in column $R^{(m)}$ of R (since $R^{(m)} = M \circ T^{(m)}$ and M is Vandermonde) for $m \in [0, \dots, \ell k - 1]$. Here $\circ$ denotes the matrix multiplication.
 3. Parties output the $(\ell k)^2$ elements of R as final randomness.

Alternative output: if every party in $\mathbb{C}$ has opened her secrets correctly in step *Reveal*, then:

1. Parties compute $R = M \circ T$ in the following way: Let S be $(n - t) \times (\ell k)$ matrix with rows indexed by the parties in $\mathbb{C}$ and where the row corresponding to P_m is $(g^{f_{-(\ell-1),m}^{(1)}}, \dots, g^{f_{0,m}^{(1)}}), \dots, (g^{f_{-(\ell-1),m}^{(k)}}, \dots, g^{f_{0,m}^{(k)}})$. Then, each party computes $U = M \cdot S \in \mathbb{Z}_q^{\ell k \times k\ell}$ (using the standard FFT in $\mathbb{Z}_q$ to compute each column) and $R = g^U$, meaning the (i, j) -th element in R is g^y , where y is the (i, j) -th element in U .
2. Parties output the $(\ell k)^2$ elements of R as final randomness.

Security. The security of ALBATROSS+ inherits the guarantees of the original ALBATROSS protocol, as we only replace the packed II_S with BII_S , a batched and packed variant. This preserves the same security properties while enabling batching k executions of the protocol.

Efficiency. Our hybrid approach achieves an asymptotic reduction in protocol executions, lowering amortized communication complexity to $O(1)$ compared to the $O(k)$ overhead of naively repeating the base protocol k times. Computationally, as shown in Table 2, ALBATROSS+ outperforms ALBATROSS by $\sim 50\%$ in both the commit and reveal phases. These efficiency gains are achieved without compromising security, retaining the integrity guarantees of the original protocol, making it even more efficient for high-throughput distributed randomness generation with a limited number of parties.

8 Conclusion

We introduced a tunable batched and packed extension of the unified framework Π [4], enabling efficient (publicly) verifiable secret sharing protocols based on Shamir sharing in the synchronous model. Like the original framework, our variant requires only a random oracle and a secure (basic vector) commitment scheme, without homomorphism requirements. Tunable parameters let users balance robustness and efficiency as needed. Using this generalized construction, we developed batched and packed versions of all VSS schemes from [4], achieving significant efficiency improvements for large-scale secret sharing. Our hash-based construction is especially promising for post-quantum threshold protocols, and we also proposed batched variants of Feldman’s [24] and Pedersen’s [32] VSSs.

To construct a more efficient perfectly simulatable batched VSS scheme, providing information-theoretic privacy against unbounded adversaries, we proposed a novel and simple perfectly-hiding vector commitment scheme in the random oracle model that outperforms the vector variant of Pedersen commitment significantly. We anticipate independent applications for this re-randomizable commitment scheme beyond VSS.

Building on our framework, we further proposed a tunable batched and packed *publicly* verifiable secret sharing scheme, generalizing the packed PVSS construction in [4, 14]. Key to this result is a batched generalization of Baghery’s Schnorr-style protocol [4], which allows proving knowledge of multiple secret polynomials at nearly the same cost as a single instance in the non-batched case. As a concrete application, we improved the ALBATROSS randomness generation protocol [14] using our batched and packed PVSS scheme. We believe the new batched PVSS scheme can be used in various other applications like those discussed in [34].

Our implementation results confirm that the new BP-VSS and BP-PVSS protocols are highly efficient for large-scale applications, both in computation and communication. In particular, when sharing many secrets, the BP-VSS schemes BII_{LA} and $\text{BII}_{\text{P}+}$ and the BP-PVSS scheme BII_{S} achieve performance nearly identical to plain Shamir sharing in the distribution phase. Our hash-based batched VSS scheme holds particular significance for lattice and isogeny-based threshold cryptography. In CSIDH-based protocols [18], such as DKGs and threshold signatures [1, 2, 7], parties must verifiably share large numbers of secrets (e.g., $k > 2^{10}$ in CSI-FiSh [8]). Repeated invocations of standard VSS

for k secrets impose prohibitive overheads, making our batched approach essential for practical PQ deployments. By enabling simultaneous distribution and verification of multiple secrets, our framework addresses a critical performance bottleneck in real-world post-quantum secure threshold protocols.

Finally, we highlight that the tunable batched-and-packed nature of new framework provides a versatile tool for constructing efficient VSS variants and re-engineering threshold protocols. While detailed exploration of these applications lies beyond this work, we highlight promising directions for future research, including integration with advanced distributed protocols and further optimizations for emerging post-quantum primitives.

Acknowledgments

This work was supported by the Flemish Government through the Cybersecurity Research Program with grant number VOEWICS02 and Research grant VIL53029 from VILLUM FONDEN.

References

1. Shahla Atapoor, Karim Bagheri, Daniele Cozzo, and Robi Pedersen. Practical robust DKG protocols for CSIDH. In Mehdi Tibouchi and Xiaofeng Wang, editors, *ACNS 23: 21st International Conference on Applied Cryptography and Network Security, Part II*, volume 13906 of *Lecture Notes in Computer Science*, pages 219–247, Kyoto, Japan, June 19–22, 2023. Springer, Cham, Switzerland.
2. Shahla Atapoor, Karim Bagheri, Daniele Cozzo, and Robi Pedersen. VSS from distributed ZK proofs and applications. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings*, *Lecture Notes in Computer Science*. Springer, 2023.
3. Michael Backes, Aniket Kate, and Arpita Patra. Computational verifiable secret sharing revisited. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology - ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 590–609, Seoul, South Korea, December 4–8, 2011. Springer Berlin Heidelberg, Germany.
4. Karim Bagheri. A unified framework for computational verifiable secret sharing. In Tibor Jager and Jiaxin Pan, editors, *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Roros, Norway May 12-15, 2025, Proceedings*, *Lecture Notes in Computer Science*.
5. Mihir Bellare, Juan A. Garay, and Tal Rabin. Fast batch verification for modular exponentiation and digital signatures. In Kaisa Nyberg, editor, *Advances in Cryptology - EUROCRYPT'98*, volume 1403 of *Lecture Notes in Computer Science*, pages 236–250, Espoo, Finland, May 31 – June 4, 1998. Springer Berlin Heidelberg, Germany.
6. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract).

- In *20th Annual ACM Symposium on Theory of Computing*, pages 1–10, Chicago, IL, USA, May 2–4, 1988. ACM Press.
7. Ward Beullens, Lucas Disson, Robi Pedersen, and Frederik Vercauteren. CSI-RASh: Distributed key generation for CSIDH. In Jung Hee Cheon and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20-22, 2021, Proceedings*, volume 12841 of *Lecture Notes in Computer Science*, pages 257–276. Springer, 2021.
 8. Ward Beullens, Thorsten Kleinjung, and Frederik Vercauteren. CSI-FiSh: Efficient isogeny based signatures through class group computations. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part I*, volume 11921 of *Lecture Notes in Computer Science*, pages 227–247, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland.
 9. G. R. Blakley. Safeguarding cryptographic keys. In *1979 International Workshop on Managing Requirements Knowledge, MARK 1979, New York, NY, USA, June 4-7, 1979*, pages 313–318. IEEE, 1979.
 10. G. R. Blakley and Catherine Meadows. Security of ramp schemes. In G. R. Blakley and David Chaum, editors, *Advances in Cryptology, Proceedings of CRYPTO '84, Santa Barbara, California, USA, August 19-22, 1984, Proceedings*, volume 196 of *Lecture Notes in Computer Science*, pages 242–268. Springer, 1984.
 11. Jonathan Bootle and Jens Groth. Efficient batch zero-knowledge arguments for low degree polynomials. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018: 21st International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 10770 of *Lecture Notes in Computer Science*, pages 561–588, Rio de Janeiro, Brazil, March 25–29, 2018. Springer, Cham, Switzerland.
 12. Ignacio Cascudo, Daniele Cozzo, and Emanuele Giunta. Verifiable secret sharing from symmetric key cryptography with improved optimistic complexity. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part VII*, volume 15490 of *Lecture Notes in Computer Science*, pages 100–128, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
 13. Ignacio Cascudo and Bernardo David. SCRAPE: Scalable randomness attested by public entities. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 17: 15th International Conference on Applied Cryptography and Network Security*, volume 10355 of *Lecture Notes in Computer Science*, pages 537–556, Kanazawa, Japan, July 10–12, 2017. Springer, Cham, Switzerland.
 14. Ignacio Cascudo and Bernardo David. ALBATROSS: Publicly Attestable Batched Randomness based On Secret Sharing. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 311–341, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.
 15. Ignacio Cascudo and Bernardo David. ALBATROSS: publicly attestable batched randomness based on secret sharing. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 311–341. Springer, 2020.
 16. Ignacio Cascudo and Bernardo David. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part V*, volume 14655 of *Lecture Notes in Computer Science*, pages 216–248, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.

17. Ignacio Cascudo, Bernardo David, Lydia Garms, and Anders Konring. YOLO YOSO: Fast and simple encryption and secret sharing in the YOSO model. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 651–680, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland.
18. Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: An efficient post-quantum commutative group action. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part III*, volume 11274 of *Lecture Notes in Computer Science*, pages 395–427, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland.
19. Dario Catalano and Dario Fiore. Vector commitments and their applications. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *Public-Key Cryptography - PKC 2013 - 16th International Conference on Practice and Theory in Public-Key Cryptography, Nara, Japan, February 26 - March 1, 2013. Proceedings*, volume 7778 of *Lecture Notes in Computer Science*, pages 55–72. Springer, 2013.
20. David Chaum, Jan-Hendrik Evertse, and Jeroen van de Graaf. An improved protocol for demonstrating possession of discrete logarithms and some generalizations. In David Chaum and Wyn L. Price, editors, *Advances in Cryptology – EUROCRYPT’87*, volume 304 of *Lecture Notes in Computer Science*, pages 127–141, Amsterdam, The Netherlands, April 13–15, 1988. Springer Berlin Heidelberg, Germany.
21. Benny Chor, Shafi Goldwasser, Silvio Micali, and Baruch Awerbuch. Verifiable secret sharing and achieving simultaneity in the presence of faults (extended abstract). In *26th Annual Symposium on Foundations of Computer Science*, pages 383–395, Portland, Oregon, October 21–23, 1985. IEEE Computer Society Press.
22. Ronald Cramer, Ivan Damgård, and Ueli M. Maurer. General secure multi-party computation from any linear secret-sharing scheme. In Bart Preneel, editor, *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 316–334, Bruges, Belgium, May 14–18, 2000. Springer Berlin Heidelberg, Germany.
23. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *Advances in Cryptology - CRYPTO ’89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 307–315. Springer, 1989.
24. Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science*, pages 427–437, Los Angeles, CA, USA, October 12–14, 1987. IEEE Computer Society Press.
25. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany.
26. Matthew K. Franklin and Moti Yung. Communication complexity of secure computation (extended abstract). In S. Rao Kosaraju, Mike Fellows, Avi Wigderson, and John A. Ellis, editors, *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada*, pages 699–710. ACM, 1992.

27. Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, January 2007.
28. Rosario Gennaro, Michael O. Rabin, and Tal Rabin. Simplified VSS and fast-track multiparty computations with applications to threshold cryptography. In Brian A. Coan and Yehuda Afek, editors, *17th ACM Symposium Annual on Principles of Distributed Computing*, pages 101–111, Puerto Vallarta, Mexico, June 28 – July 2, 1998. Association for Computing Machinery.
29. Craig Gentry, Shai Halevi, and Vadim Lyubashevsky. Practical non-interactive publicly verifiable secret sharing with thousands of parties. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 458–487, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
30. Jens Groth. Non-interactive distributed key generation and key resharing. Cryptology ePrint Archive, Report 2021/339, 2021.
31. Alireza Kavousi, Zhipeng Wang, and Philipp Jovanovic. Sok: Public randomness. In *9th IEEE European Symposium on Security and Privacy, EuroS&P 2024, Vienna, Austria, July 8-12, 2024*, pages 216–234. IEEE, 2024.
32. Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *Advances in Cryptology – CRYPTO’91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140, Santa Barbara, CA, USA, August 11–15, 1992. Springer Berlin Heidelberg, Germany.
33. Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO’89*, volume 435 of *Lecture Notes in Computer Science*, pages 239–252, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.
34. Berry Schoenmakers. A simple publicly verifiable secret sharing scheme and its application to electronic. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO’99*, volume 1666 of *Lecture Notes in Computer Science*, pages 148–164, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany.
35. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
36. Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.
37. Victor Shoup and Nigel P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. Cryptology ePrint Archive, Paper 2023/536, 2023. <https://eprint.iacr.org/2023/536>.
38. Victor Shoup and Nigel P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. *Journal of Cryptology*, 37(3):27, July 2024.
39. Jinhyun So, Basak Güler, and Amir Salman Avestimehr. Byzantine-resilient secure federated learning. *IEEE J. Sel. Areas Commun.*, 39(7):2168–2181, 2021.
40. Alin Tomescu, Robert Chen, Yiming Zheng, Ittai Abraham, Benny Pinkas, Guy Golan-Gueta, and Srinivas Devadas. Towards scalable threshold cryptosystems. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*, pages 877–893. IEEE, 2020.

Appendix:

A Preliminaries

Lemma 1 (Schwartz-Zippel Lemma). *In any field $\mathbb{F}$, let $f : \mathbb{F}^m \rightarrow \mathbb{F}$ be a non-zero m -variate polynomial of total degree at most d . Then for any finite set $S \subseteq \mathbb{F}$ and m random samples $x_1, \dots, x_m \leftarrow S$,*

$$\Pr[f(x_1, \dots, x_m) = 0] \leq \frac{d}{|S|}.$$

In other words, the probability that f evaluates to zero on a random evaluation point is bounded by $d/|S|$.

Reed-Solomon codes. For a finite field $\mathbb{F}$, n evaluation points $i_1, i_2, \dots, i_n \in \mathbb{F}$ and an integer d , we define an (n, d) -Reed-Solomon code as

$$\{(f(i_1), f(i_2), \dots, f(i_n)) \mid f \in \mathbb{F}[x]_{d-1}\}.$$

A.1 Vector Commitments

Vector commitments allow a committer to commit to a vector of messages in a single operation. We summarize the (basic) definition of *vector commitments*, which is a simplified version of the one presented by Catalano and Fiore [19].

Definition 2 (Vector Commitments). *A basic vector commitment scheme over a message space $\mathcal{M}$ is a tuple of algorithms $\text{VC} = (\text{Setup}, \mathcal{C}, \text{Verify})$ defined as follows:*

1. $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: *On input the security parameter 1^λ , this algorithm outputs public parameters pp . All subsequent algorithms implicitly take pp as input, but we omit it for notational clarity.*
2. $\mathcal{C}(m_1, \dots, m_k; r) \rightarrow \text{com}$: *On input a vector $(m_1, \dots, m_k) \in \mathcal{M}^k$ and a randomizer r , this algorithm outputs a commitment com .*
3. $\text{Verify}(\text{com}, m_1, \dots, m_k, r) \rightarrow \{0, 1\}$: *On input a commitment com , a vector $(m_1, \dots, m_k)$, and a value r , this algorithm outputs 1 (accept) or 0 (reject).*

A basic vector commitment scheme must satisfy the following properties:

- **Correctness.** For security parameter λ and any vector $(m_1, \dots, m_k) \in \mathcal{M}^k$, a vector commitment scheme satisfies *perfect correctness* if:

$$\Pr \left[\text{pp} \leftarrow \text{Setup}(1^\lambda), r \leftarrow \{0, 1\}^\lambda, \text{com} \leftarrow \mathcal{C}(m_1, \dots, m_k; r) : \text{Verify}(\text{com}, m_1, \dots, m_k, r) = 1 \right] = 1.$$

- **Binding.** For security parameter λ , a basic vector commitment scheme satisfies *perfect binding* if for any unbounded adversary $\mathcal{A}$:

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{com}, (\vec{m}, r), (\vec{m}', r')) \leftarrow \mathcal{A}(\text{pp}) : \\ \vec{m}, \vec{m}' \in \mathcal{M} \wedge ((\vec{m} \neq \vec{m}') \vee (r \neq r')) \wedge \\ \text{Verify}(\text{com}, \vec{m}, r) = 1 \wedge \text{Verify}(\text{com}, \vec{m}', r') = 1 \end{array} \right] = \text{negl}(\lambda).$$

For *computational* binding, this must hold against PPT adversaries.

- **Hiding.** For security parameter λ , a basic vector commitment scheme satisfies *perfect* hiding if for any unbounded adversary $\mathcal{A}$:

$$\left| \Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda), (\vec{m}_0, \vec{m}_1) \leftarrow \mathcal{A}(\text{pp}), b \leftarrow_{\$} \{0, 1\}, \\ r \leftarrow_{\$} \{0, 1\}^\lambda, \text{com} \leftarrow \mathcal{C}(\vec{m}_b; r), b' \leftarrow \mathcal{A}(\text{com}) : b = b' \end{array} \right] - \frac{1}{2} \right| = \text{negl}(\lambda).$$

For *computational* hiding, this must hold against PPT adversaries.

A.2 Sigma Protocols and Relevant Constructions

In this section we recall the definition of sigma protocols (Σ -protocols) and review a few relevant constructions. Let $X = X(\lambda)$ and $W = W(\lambda)$ be sets. Let R be a relation on $X \times W$ that defines a language $L = \{x \in X : \exists w \in W, R(x, w) = 1\}$. Given $x \in L$, an element $w \in W$ such that $R(x, w) = 1$ is called a witness. Let relation generator $\mathcal{R}$ be a PPT algorithm such that $\mathcal{R}(1^\lambda)$ outputs pairs (x, w) such that $R(x, w) = 1$.

A public-coin sigma-protocol (Σ -protocol) for the relation R is a 3-round interactive protocol between two Probabilistic Polynomial Time (PTT) algorithms: a prover P and a verifier V . P holds a witness w for $x \in L$ and V is given x . In first round, P sends a commitment value a to V , and then in second round, V answers with a randomly sample challenge value d . Finally, P answers with a response z , and V verifies the proof and outputs **true** or **false**. The triple $\text{trans} := (a, d, z)$ is called a transcript of the Σ -protocol. A Σ -protocol is supposed to satisfy the properties of *Completeness*, *Honest-Verifier Zero-Knowledge* (HVZK), and *Special Soundness*, as defined below.

Definition 3 (Completeness). A Σ -protocol with parties (P, V) is complete for $\mathcal{R}$, if for all $(x, w) \in R$, the honest V will always accept the honest P .

Definition 4 (HVZK). A Σ -protocol with parties (P, V) satisfies HVZK for $\mathcal{R}$, if there exists a PPT algorithm $\mathcal{S}$ that given $x \in X$, can simulate the trans of the scheme, s.t. for all $x \in L$, $(x, w) \in R$,

$$\text{trans}(P(x, w) \leftrightarrow V(x)) \approx \text{trans}(\mathcal{S}(x))$$

where $\text{trans}(P(\cdot) \leftrightarrow V(\cdot))$ indicates the transcript of the Σ -protocol with (P, V) , and $\approx$ denotes the indistinguishability of transcripts.

Definition 5 (m -special Soundness). A Σ -protocol with parties (P, V) is m -special sound for $\mathcal{R}$, if there exists a PPT extractor $\mathcal{E}$, such that for any $x \in L$, given m valid transcripts $(a, d_1, z_1), \dots, (a, d_m, z_m)$ for the same message a but different $d_1, \dots, d_m$, then $\mathcal{E}(a, d_1, z_1, \dots, d_m, z_m)$ outputs a witness w for R .

Using Fiat-Shamir transform [25], a public-coin, complete, HVZK, and special soundness Σ -protocol can be turned into a Non-Interactive Zero-Knowledge (NIZK) proof or argument of knowledge.

A.3 A NIZK for Simultaneous Solution of Multiple DL Instance

Let $\mathbb{G}$ be a group with hard DL, and $g_1, g_2, \dots, g_k$ be k group generators where the link between any pair of them is unknown. Let a prover aim to convince a verifier that for the public statement $(g_1, g_2, \dots, g_k, x_1, x_2, \dots, x_k)$, he knows a witness w which holds in the following relation,

$$R_{k\text{-DLEQ}} = \{(g_1, \dots, g_k, x_1, \dots, x_k), w \mid x_1 = g_1^w \wedge \dots \wedge x_k = g_k^w\}. \quad (1)$$

For $k = 2$, this relation is known as DL EQuality (DLEQ). In [20], Chaum, Evertse, and Graaf introduced an efficient NIZK proof of knowledge for k -DLEQ, as summarized in Fig. 8. This protocol is used in the reconstruction phase of our proposed batched publicly verifiable secret sharing scheme.

Prover: Given the statement $(g_1, \dots, g_k, x_1, \dots, x_k) \in \mathbb{G}$ and the witness value $w \in \mathbb{Z}_q$, proceed as follows and output a proof π. 1. Sample $r \leftarrow \mathbb{Z}_q$ uniformly at random; and set $c_i = g_i^r$ for $i = 1, \dots, k$. 2. Set $d \leftarrow \mathcal{H}(x_1, \dots, x_k, c_1, \dots, c_k)$, where $\mathcal{H}$ is a random oracle. 3. Set $z = r + d \cdot w \pmod q$; and Return $\pi := (d, z)$ Verifier: Given the statement $(g_1, \dots, g_k, x_1, \dots, x_k) \in \mathbb{G}$ and the proof $\pi = (d, z)$, checks if $d = \mathcal{H}(x_1, \dots, x_k, \frac{g_1^z}{x_1^d}, \dots, \frac{g_k^z}{x_k^d})$ and outputs true or false.
--

Fig. 8. A NIZK proof of knowledge for $R_{k\text{-DLEQ}}$ [20].

A.4 A NIZK Proof of Knowledge for the PDL Problem

In [4, Sec. 5], Bagheri introduced the Polynomial Discrete Logarithm (PDL) problem as a generalization of the standard DL assumption. The standard DL problem corresponds to the special case where $(n, t) = (1, 0)$.

Definition 6 (Polynomial DL Problem). Let $\mathbb{G}$ be a finite cyclic group of order q generated by g . Given $F_1, \dots, F_n$ from $\mathbb{G}$ and distinct elements $x_1, \dots, x_n$ from $\mathbb{Z}_q$, find the polynomials $f(X)$ of (at most) degree t , where $0 \leq t \leq n-1$, such that $F_i = g^{f(x_i)}$ for all $i = 1, \dots, n$. In other words, an algorithm $\mathcal{A}$ has advantage ϵ in solving PDL in $\mathbb{G}$ if

$$\Pr[\mathcal{A}(x_1, \dots, x_n, g, g^{f(x_1)}, \dots, g^{f(x_n)}) = f(X)] \geq \epsilon$$

where $f(X) \in \mathbb{Z}_q[X]_t$ is (at most) degree- t witness polynomial with $0 \leq t \leq n-1$, and the probability is over the random choice of generator $g \in \mathbb{G}^*$ and the distinct choice of $x_1, \dots, x_n$ in $\mathbb{Z}_q$. We say that the (t, ϵ) -PDL assumption holds in $\mathbb{G}$ if no t -time algorithm has advantage at least ϵ in solving the PDL problem in $\mathbb{G}$.

In [4], Bagheri also generalized Schnorr's identification protocol [33] and proposed an efficient Σ -protocol that enables a prover to demonstrate knowledge of a solution to the PDL problem. The non-interactive variant of this protocol, as presented in [4, Sec. 5], is shown in Fig. 9.

Prover: Given the statement $(g, x_i, F_i)_{i=1}^n$ and the witness polynomials $f(X)$, proceed as follows and output a proof π .

1. Sample a degree- t polynomial $r(X) \in \mathbb{Z}_q[X]_t$; Set $\{\Gamma_i = g^{r(x_i)}\}_{i=1}^n$.
2. Set $d \leftarrow \mathcal{H}(F_1, \dots, F_n, \Gamma_1, \dots, \Gamma_n)$, where $\mathcal{H}$ is a random oracle.
3. Set $z(X) = r(X) + df(X) \pmod{q}$
4. Return $\pi := (d, z(X))$

Verifier: Given statement $(g, \{x_i, F_i\}_{i=1}^n)$ and $\pi := (d, z(X))$, the verifier first check if $z(X)$ is a degree- t polynomial. If so, then it checks if

$$d = \mathcal{H}(F_1, \dots, F_n, \frac{g^{z(x_1)}}{F_1^d}, \dots, \frac{g^{z(x_n)}}{F_n^d})$$

and outputs **true** or **false**.

Fig. 9. A NIZK proof of knowledge for the PDL relation [4].

B An Overview of Π

The Π framework [4] leverages distributed ZK proofs to add verifiability to Shamir's secret sharing. This is achieved using a (not necessarily homomorphic) commitment scheme together with a random oracle. Given the parameters (n, t) , a random oracle $\mathcal{H}$, and a secret value f_0 , the dealer proceeds as follows:

1. It samples two random polynomials $f(x), r(x) \in \mathbb{Z}_q[x]$ of degree t such that $f(0) = f_0$ (the secret) and $r(0)$ is uniformly random.
2. For each party P_i ($i = 1, \dots, n$), it computes $f_i := f(i)$ (shares) and $r_i := r(i)$, and commits to each pair using a commitment scheme: $c_i := \mathcal{C}(f_i, r_i)$, or optionally using vector commitment $\mathcal{C}$ and auxiliary randomness γ_i as $c_i := \mathcal{C}((f_i, r_i), \gamma_i)$.
3. Next, it computes a challenge value d using the random oracle $\mathcal{H}$, as $d := \mathcal{H}(c_1, \dots, c_n)$, and computes the linear combination of the two polynomials $z(x) := r(x) + d \cdot f(x)$.
4. At the end, it broadcasts the tuple $\pi_{share} := (c_1, \dots, c_n, z(X))$ along with sending each share (f_i, γ_i) privately to party P_i .

In order to verify the shares, each party P_i verifies whether $z(x)$ is a degree- t polynomial and whether the commitment $c_i \stackrel{?}{=} \mathcal{C}(f_i, z(i) - d \cdot f_i)$, or with auxiliary randomness, $c_i \stackrel{?}{=} \mathcal{C}((f_i, z(i) - d \cdot f_i), \gamma_i)$. If this check fails, the party issues a complaint. If more than t parties complain, the dealer is disqualified. This mechanism ensures that shares are consistent and correctly distributed.

C Security Proofs

C.1 The Proof of Theorem 1

Proof. Let us first discuss the simulateability of the scheme. This follows from the hiding property of the commitment scheme $\mathcal{C}$ and the fact that any t secret shares from some polynomial $f^{(j)}$ reveal nothing about the packed secrets

$f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)}$. In other words, one could simulate the view of the adversary with access to only the secret shares of t different parties $\{\tilde{f}_i^{(j)}\}_{i=1, j=1}^{t, k}$. The simulator samples random degree $t + \ell$ polynomials $\tilde{r}(X)$ and $\tilde{f}^{(j)}(X)$ such that $\tilde{f}^{(j)}(i) = \tilde{f}_i^{(j)}$ for $j = 1, \dots, k$ and $i = 1, \dots, t$. Then, it uses the hiding commitment scheme to compute $\tilde{c}_i = \mathcal{C}((\tilde{f}^{(1)}(i), \dots, \tilde{f}^{(k)}(i), \tilde{r}(i)), \gamma_i)$ for $i = 1, \dots, n$. Using the challenge values $\tilde{d}_1, \dots, \tilde{d}_k$, the simulator computes a simulated $\tilde{z}(X) := \tilde{r}(X) + \sum_{j=1}^k \tilde{d}_j \tilde{f}^{(j)}(X)$. The simulated transcript is constructed as $\tilde{\pi}_{share} := (\tilde{c}_1, \dots, \tilde{c}_n, \tilde{z}(X))$. Since the adversary could produce a perfect simulation of the public transcript, it reveals nothing about the secret shares $\{(f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)})_{j=1}^k\}$ for up to t malicious parties. Next, we prove that the scheme satisfies verifiability with unique reconstruction.

Since we assume a packing size ℓ and we have passed the verification stage, we can assume WLOG that at least the parties $P_1, \dots, P_{t+\ell}$ are in possession of k secrets $\{f_i^{(j)}\}_{i=1, j=1}^{t+\ell, k}$ such that $c_i = \mathcal{C}((\{f_i^{(j)}\}_{j=1}^k, z(i) - \sum_{j=1}^k d_j f_i^{(j)}), \gamma_i)$ for $i = 1, \dots, t + \ell$, where $d_1, \dots, d_k$ are computed using the RO and the public commitments c_i for $i = 1, \dots, n$. These secrets can be used to construct the polynomials $f^{(1)}(X), \dots, f^{(k)}(X)$, which define $f_0^{(1)} = f^{(1)}(0), \dots, f_0^{(k)} = f^{(k)}(0)$. An honest party can use these polynomials to reconstruct the secrets of the other parties and verify whether $c_i = \mathcal{C}((\{f_i^{(j)}\}_{j=1}^k, r_i), \gamma_i)$ for all $r_i = z(i) - \sum_{j=1}^k d_j f_i^{(j)}$.

We show that at the end of the Reconstruction phase, the output is a unique set of values $F_0 = \{(f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)})_{j=1}^k\}$ with probability greater than $1 - 1/q$. We denote the set of qualified parties remaining after verification phase as $\hat{P} = \{P_{\hat{i}_i}\}_{i=1}^{\hat{n}}$ for $\{\hat{i}_i\}_{i=1}^{\hat{n}} \subseteq [n]$ and $t + \ell \leq \hat{n} \leq n$. WLOG we can state that $\{P_{\hat{i}_i}\}_{i=1}^{t+\ell} \in \hat{P}$. Assume that some set of parties P^* of size $t + \ell$ reconstructs $\hat{F}_0 = \{(\hat{f}_{-(\ell-1)}^{(j)}, \dots, \hat{f}_0^{(j)})_{j=1}^k\} \neq F_0$ based on the values $\{\hat{f}_{\hat{i}_i}^{(j)}\}_{j=1, i=1}^{k, t+\ell}$ for $\{\hat{i}_i\}_{i=1}^{t+1}$. Define an $(\hat{n}, t + \ell)$ -Reed-Solomon code for the evaluation points $\hat{i}_1, \hat{i}_2, \dots, \hat{i}_{\hat{n}}$ and a corresponding Vandermonde matrix

$$V = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \hat{i}_1 & \hat{i}_2 & \dots & \hat{i}_{\hat{n}} \\ \hat{i}_1^2 & \hat{i}_2^2 & \dots & \hat{i}_{\hat{n}}^2 \\ \vdots & & \ddots & \vdots \\ \hat{i}_1^{\hat{n}-1} & \hat{i}_2^{\hat{n}-1} & \dots & \hat{i}_{\hat{n}}^{\hat{n}-1} \end{bmatrix}.$$

Now define a check matrix C as the rightmost $\hat{n} - t - 1$ columns of V^{-1} . Thereby a vector is an element of the code if and only if it lies in the nullspace of C .

By our assumption we know that $\hat{F}_0$ will differ from F_0 in at least one element, i.e.

$$(f_{-(\ell-1)}^{(j)}, \dots, f_0^{(j)}) \neq (\hat{f}_{-(\ell-1)}^{(j)}, \dots, \hat{f}_0^{(j)})$$

for some $j \in [k]$. Therefore, the sets $\{\hat{f}_{\hat{i}_i}^{(j)}\}_{i=1}^{t+\ell}$ and $\{f_{\hat{i}_i}^{(j)}\}_{i=1}^{t+\ell}$ must differ in at least one value. This implies that the set $f^{(j)} = \{f_1^{(j)}, \dots, f_{\hat{n}}^{(j)}\}$ (of which both are a subset) is not an $(\hat{n}, t + \ell)$ -Reed-Solomon codeword. In other words, $f^{(j)} \cdot C \neq 0$.

Therefore, we can state that for matrix $F = \{f_i^{(j)}\}_{j=1, i=1}^{k, \hat{n}}$, the matrix $F \cdot C$ is a non-zero matrix. On the other hand, since the reconstruction has not resulted in false and we assume the commitment scheme is binding, we can also state that for $\vec{z} = \{z(i_i)\}_{i=1}^{\hat{n}}$ and $\vec{d} = \{d_1, \dots, d_k\}$, the vector $\vec{z} - \vec{d} \cdot \hat{F}$ (whose values all lie on $\hat{r}(x)$) must be an $(\hat{n}, t + \ell)$ -Reed-Solomon codeword. This implies that

$$(\vec{z} - \vec{d} \cdot F) \cdot C = \vec{z} \cdot C - \vec{d} \cdot F \cdot C = 0.$$

For a random $\vec{d} = \{d_1, \dots, d_k\}$ we bound the probability that the above holds. Since $\mathbb{F} \cdot C$ is a non-zero matrix, it contains a non-zero column $\vec{a}$ of length k . So there must be an element of $\vec{z} \cdot C$, which we note as b , such that

$$b - \langle \vec{d}, \vec{a} \rangle = 0,$$

The Schwartz-Zippel lemma states that this holds with probability at most $1/q$. Note that for $k = 1$ and $l = 1$, the protocol in Figure 2 reduces to the protocol introduced by Baghery [4]. In this case, the proof reduces to the contradiction that $F = \{f_i^{(1)}\}_{i=1}^{\hat{n}}$ is not a codeword while $\vec{z} - d \cdot \vec{F}$ is a codeword in the $(\hat{n}, t + 1)$ -Reed-Solomon code. This clearly contradicts the linearity of Reed-Solomon codes. $\square$

C.2 The Proof of Theorem 2

Proof. Given the hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$, which is modeled as a random oracle, we analyze the security properties. For the correctness, we observe that given $(m_1, \dots, m_k, r)$, the commitment process is deterministic: 1) Hash computation $m' = \mathcal{H}(m_1, \dots, m_k)$ is deterministic. 2) Pedersen commitment $g_1^{m'} g_2^r$ is perfectly correct. Thus for any $r \leftarrow \$\mathbb{Z}_q$:

$$\Pr [\text{Verify}(\mathcal{C}(m_1, \dots, m_k; r), m_1, \dots, m_k, r) = 1] = 1.$$

Perfect Hiding. The perfectly hiding property follows from the Pedersen commitment. For any two message vectors $\vec{m}_0, \vec{m}_1 \in \mathcal{M}^k$, the commitment:

$$\mathcal{C}(\vec{m}_b; r) = g_1^{\mathcal{H}(\vec{m}_b)} g_2^r = g_1^{h_b} g_2^r$$

Since r is uniformly random and g_2^r is a uniformly random group element:

$$\forall m'_0, m'_1 \in \mathbb{Z}_q : g^{m_0} h^r \equiv g^{m'_1} h^{r'}$$

where r, r' are both uniformly random. Hence the distributions are identical, providing perfect hiding.

Computational Binding. Suppose there exists a PPT adversary $\mathcal{A}$ that breaks binding with non-negligible probability. Then $\mathcal{A}$ can produce:

$$(\vec{m}, r) \neq (\vec{m}', r') \quad \text{s.t.} \quad g_1^{\mathcal{H}(\vec{m})} g_2^r = g_1^{\mathcal{H}(\vec{m}')} g_2^{r'}$$

This implies:

$$g_1^{\Delta M} = g_2^{\Delta r} \quad \text{where} \quad \begin{cases} \Delta M = \mathcal{H}(\vec{m}) - \mathcal{H}(\vec{m}') \\ \Delta r = r' - r \end{cases}$$

We analyze two cases:

Case 1: $\Delta M = 0 \pmod{q}$. This implies $g_2^{\Delta r} = 1$, hence $\Delta r = 0 \pmod{q}$. But since $(\vec{m}, r) \neq (\vec{m}', r')$, this requires $\vec{m} \neq \vec{m}'$ while $\mathcal{H}(\vec{m}) = \mathcal{H}(\vec{m}')$, a hash collision. This contradicts the collision resistance of $\mathcal{H}$ in the RO model.

Case 2: $\Delta M \neq 0 \pmod{q}$. We can compute:

$$g_2 = g_1^{\Delta M \cdot (\Delta r)^{-1}}$$

revealing the discrete logarithm $\log_{g_1} g_2 = \Delta M \cdot (\Delta r)^{-1} \pmod{q}$. This contradicts the DL assumption. Since both cases lead to contradiction, the scheme must be computationally binding. $\square$

C.3 The Proof of Theorem 3

Proof. The proofs of *completeness* and *verifiability* follow directly from the general construction in Theorem 1. We focus on proving *unpredictability* via a reduction to the DL problem.

Assume an adversary $\mathcal{A}$, given π_{share} and the shares of t corrupted parties (i.e., $\{\{f_i^{(j)}\}_{j=1}^k\}_{i \in Q}$ where $|Q| \leq t$), can recover at least one of the $k \times \ell$ shared secrets. Without loss of generality, suppose $\mathcal{A}$ extracts $f_0^{(1)}$. We construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break the DL assumption.

For concreteness, we demonstrate the reduction for $(n, t, k, \ell) = (5, 1, 2, 2)$, where $n \geq 2t + \ell$. Here, the dealer shares $\ell \times k = 4$ secrets $f_{-1}^{(1)}, f_0^{(1)}, f_{-1}^{(2)}, f_0^{(2)}$ by packing $\ell = 2$ secrets into each of $k = 2$ random polynomials of degree $t + \ell - 1 = 2$.

Adversary $\mathcal{B}$'s strategy:

1. *Receive DL challenge:* $\mathcal{B}$ is given $(g_1, y_{01} = g_1^{f_0^{(1)}})$ and aims to compute $f_0^{(1)}$.
2. *Setup group generators:* Sample $\alpha, \beta \xleftarrow{\$} \mathbb{Z}_q$. Set $g_2 = g_1^\alpha$ and $g_3 = g_2^\beta$.
3. *Simulate shares for polynomial $f^{(1)}(x)$:* Sample $f_1^{(1)}, f_2^{(1)} \xleftarrow{\$} \mathbb{Z}_q$. Compute $y_{11} = g_1^{f_1^{(1)}}$ and $y_{21} = g_1^{f_2^{(1)}}$. Use Lagrange interpolation in the exponent to compute $y_{31} = g_1^{f_3^{(1)}}$, $y_{41} = g_1^{f_4^{(1)}}$, and $y_{51} = g_1^{f_5^{(1)}}$ from (y_{01}, y_{11}, y_{21}) .
4. *Simulate shares for polynomial $f^{(2)}(x)$:* Sample a random degree-2 polynomial $f^{(2)}(x)$. Compute $f_i^{(2)} = f^{(2)}(i)$ and $y_{i2} = g_2^{f_i^{(2)}}$ for $i \in \{1, \dots, 5\}$.
5. *Construct commitments:* Sample a random polynomial $z(x)$ of degree $t + \ell - 1 = 2$ and challenges $d_1, d_2 \xleftarrow{\$} \mathbb{Z}_q$. Compute for each $i \in \{1, \dots, 5\}$:

$$c_i = y_{i1} \cdot y_{i2} \cdot \frac{g_3^{z(i)}}{(y_{i1})^{\alpha\beta d_1} \cdot (y_{i2})^{\beta d_2}}.$$

6. *Program random oracle*: Set $\mathcal{H}(c_1, \dots, c_5) = (d_1, d_2)$.
7. *Interact with $\mathcal{A}$* : Send $\pi_{\text{share}} = (g_1, g_2, g_3, c_1, \dots, c_5, z(x))$ and the shares $(f_2^{(1)}, f_2^{(2)})$ to $\mathcal{A}$. Assume party 2 is corrupted (without loss of generality).
8. *Extract solution*: If $\mathcal{A}$ outputs $f_0^{(1)}$, $\mathcal{B}$ returns $f_0^{(1)}$ to solve the DL challenge.

The simulation is indistinguishable from a real execution because:

- The group elements (g_2, g_3) are distributed identically to random generators.
- Commitments $(c_1, \dots, c_5)$ are randomized by $z(x)$.
- The random oracle response (d_1, d_2) is consistent with adversarial view.

Since $\mathcal{B}$ succeeds with the same probability as $\mathcal{A}$, unpredictability holds under the DL assumption. Thus, the BP-VSS scheme BP_F satisfies the mentioned three security properties under the DL assumption. $\square$

C.4 The Proof of Theorem 4

Proof. We first prove the security of the interactive case, and then using standard Fiat-Shamir transform, extend it to the non-interactive case in the RO model.

Completeness. If the prover and verifier honestly follow the protocol, for $i = 1, \dots, n$ we have $d \leftarrow \mathcal{H}(\{F_1^{(j)}, \dots, F_n^{(j)}\}_{j=1}^k, \Gamma_1, \dots, \Gamma_n)$ and

$$g^{z(\alpha_i)} = g^{r(\alpha_i) + \sum_{j=1}^k d^j f^{(j)}(\alpha_i)} = g^{r(\alpha_i)} \cdot \prod_{j=1}^k (g^{f^{(j)}(\alpha_i)})^{d^j} = \Gamma_i \prod_{j=1}^k (F_i^{(j)})^{d^j}.$$

Honest Verifier Zero-Knowledge (HVZK): Next, we show that given the statement $(g, \alpha_1, \dots, \alpha_n, \{F_1^{(j)}, \dots, F_n^{(j)}\}_{j=1}^k)$ and the challenge values d , a simulator can simulate the transcript of the protocol. To this end, it first randomly samples a degree- t polynomial $z'(x) \in \mathbb{Z}_q[x]_t$. Then, for $i = 1, \dots, n$: sets

$$\Gamma'_i = \frac{g^{z'(\alpha_i)}}{\prod_{j=1}^k (F_i^{(j)})^{d^j}}.$$

Finally, the simulator returns $(\Gamma'_1, \dots, \Gamma'_n, d, z'(x))$ as the simulated proof and reprograms $\mathcal{H}$ to return d on input $(\{F_1^{(j)}, \dots, F_n^{(j)}\}_{j=1}^k, \Gamma_1, \dots, \Gamma_n)$. As it can be seen, since $z'(x)$ is sampled randomly, therefore $\{\Gamma'_i\}_{i=1}^n$ are also random, and the simulated proof is indistinguishable from the real one.

$(k+1)$ -Special Soundness: Let $\mathcal{E}$ be an extractor with rewinding access to the prover. In the first step of the Sigma protocol, we assume that the prover commits to $\Gamma := \{\Gamma_i\}_{i=1}^n$. Then, for a given challenge d_l , the prover responds with $z_l(X)$. We assume that the extractor rewinds the prover a total of $k+1$ times in order to extract the k different witnesses and we label these rounds $l = 0, \dots, k$. After receiving $k+1$ accepting transcripts $(\Gamma, d_l, z_l(X))$ with different d_l (Γ is the same for each round), we have that

$$g^{z_l(\alpha_i)} = \Gamma_i \prod_{j=1}^k (F_i^{(j)})^{d_l^j}$$

for all $l = 0, \dots, k$ and all $i = 1, \dots, n$. Since $F_i, F_i^{(j)} \in \mathbb{G}$, we know that there exist discrete logs with respect to g (unknown to the extractor). We label them $g^{\gamma_i} = F_i$ and $g^{\phi_i^{(j)}} = F_i^{(j)}$, so that in the exponent of g , we have the following equations

$$z_l(\alpha_i) = \gamma_i + \sum_{j=1}^k \phi_i^{(j)} d_l^j, \quad (2)$$

again for all $l = 0, \dots, k$ and all $i = 1, \dots, n$. Note that for each $i = 1, \dots, n$, we have $k + 1$ equations for the $k + 1$ unknowns $\{\gamma_i, \phi_i^{(1)}, \dots, \phi_i^{(k)}\}$. To solve this equation, we realize that we can rewrite it using the Vandermonde matrix V , which gives us the matrix equation

$$(z_0(\alpha_i), \dots, z_k(\alpha_i))^T = V(1, d_0, \dots, d_k)^T \cdot (\gamma_i, \phi_i^{(1)}, \dots, \phi_i^{(k)})^T.$$

By inverting the matrix V , we get solutions for the vector $(\gamma_i, \phi_i^{(1)}, \dots, \phi_i^{(k)})$.

We now show that for each $j = 1, \dots, k$, the extracted values $\phi_1^{(j)}, \dots, \phi_n^{(j)}$ actually define a polynomial of degree (at most) t , and therefore an instance of $R_{\text{PDL}}^{\text{Mul}}$. Let $\gamma(X)$ define the polynomial with evaluations $\gamma(\alpha_i) := \gamma_i$ and let $\phi^{(j)}(X)$ define the polynomials with $\phi^{(j)}(\alpha_i) = \phi_i^{(j)}$, each of degree n . For any fixed l , we can summarize the n equations in (2) as a polynomial equation

$$z_l(X) = \gamma(X) + \sum_{j=1}^k \phi^{(j)}(X) d_l^j.$$

Since an accepting $z_l(X)$ has degree t , this implies that all the coefficients a_i of $\gamma(X)$ and $b_i^{(j)}$ of $\phi^{(j)}(X)$ must satisfy

$$0 = a_i + \sum_{j=1}^k b_i^{(j)} d_l^j$$

for each $i = t + 1, \dots, n$. We can reinterpret $B_i(X) = a_i + \sum_{j=1}^k b_i^{(j)} X^j$ as a polynomial of degree k . Since this polynomial has to be zero for all $k + 1$ different evaluations in d_l , the only possible solution is the zero-polynomial, i.e. $a_i = b_i^{(j)} = 0$ for all $i = t + 1, \dots, n$ and all $j = 1, \dots, k$. We conclude that $\gamma(X)$ and $\phi^{(j)}(X)$ are both polynomials of degree at most t , and that therefore $\phi^{(1)}(X), \dots, \phi^{(k)}(X)$ are witnesses for $R_{\text{PDL}}^{\text{Mul}}$.

Since the interactive scheme is public coin, and satisfies completeness, $(t + 1)$ -special soundness, and (computational) HVZK, then, in the random oracle model, using Fiat-Shamir transform [25], it can be turned into a NIZK argument of knowledge scheme for $R_{\text{PDL}}^{\text{Mul}}$. $\square$

C.5 The Proof of Theorem 5

Proof. We start by showing that the reconstruction procedure towards any interpolation point α for fixed $j \in \{1, \dots, k\}$ results in a unique $g^{f^{(j)}(\alpha)}$ as long as the set of correct shares is at least $t + \ell$. Most notably, for $\alpha \in \{-(\ell - 1), \dots, 0\}$ this results in the unique reconstruction of the secrets. We argue this, by showing that the conjunction of the two proof systems employed in the **Share** and **Reconstruct** steps realize an instance of $R_{\text{PDL}}^{\text{Mul}}$.

We can show this using arguments similar to the extraction procedure of Theorem 4, where after $k + 1$ rewindings we have the equations

$$\mathbf{pk}_i^{z_i(i)} = \Gamma_i \prod_{j=1}^k \left(y_i^{(j)} \right)^{d_i^j},$$

for $i = 1, \dots, n$ and $l = 0, \dots, k$. Again, since $\Gamma_i, y_i^{(j)}, \mathbf{pk}_i \in \mathbb{G}$, we know that there must exist elements $\gamma_i, v_i^{(j)}, s_i \in \mathbb{Z}_q$, such that $g^{\gamma_i} = \Gamma_i$, $g^{v_i^{(j)}} = y_i^{(j)}$ and $g^{s_i} = \mathbf{pk}_i$, so that we can rewrite the upper equation as the following equation in the exponent of g (similar to (2)):

$$s_i z_i(i) = \gamma_i + \sum_{j=1}^k v_i^{(j)} d_i^j.$$

Now, the statement of the NIZK from Figure 8 implies that

$$\exists s_i \in \mathbb{Z}_q : \mathbf{pk}_i = g^{s_i} \wedge y_i^{(j)} = (F_i^{(j)})^{s_i}.$$

By defining $\phi_i^{(j)} = v_i^{(j)} s_i^{-1}$, such that $g^{\phi_i^{(j)}} = F_i^{(j)}$, we finally recover exactly equation (2). Using the same extraction procedure as in Theorem 4, this implies that the accepting $F_i^{(j)}$ in the reconstruction phase are indeed defined as $F_i^{(j)} = g^{f^{(j)}(i)}$, where $f^{(j)}(x) \in \mathbb{Z}_q[x]_{t+\ell-1}$. This allows efficient reconstruction of any value $g^{f^{(j)}(\alpha)}$ using Lagrange interpolation of at least $t + \ell$ accepting $F_i^{(j)}$.

We turn towards unpredictability. We first note that security of the employed encryption scheme relies on the computational Diffie-Hellman (CDH) assumption. Because, given $\mathbf{pk}_i = g^{s_i}$ and $y_i = \mathbf{pk}_i^{f_i} = g^{s_i f_i}$, an adversary able to compute g^{f_i} is exactly solving a Inverse-CDH instance $(g, g^a, g^b) \rightarrow g^{b/a}$ for $a = s_i$, $b = s_i f_i$. Inverse-CDH can be reduced to CDH by querying the Inverse-CDH twice, once with $(g, g^a, g) \rightarrow g^{a^{-1}}$ (potentially randomized) and then with $(g, g^{a^{-1}}, g^b) \rightarrow g^{ab}$. This implies that directly extracting secret information from the encryption is not possible, under the assumption that CDH is hard.

We finish this proof by showing that the view of up to t parties is simulatable. For simplicity, let these parties be $i = 1, \dots, t$. In the initialization step, the simulator samples $s_i \leftarrow \mathbb{Z}_q$ and computes $\mathbf{pk}_i = g^{s_i}$ for the parties $i = t+1, \dots, n$. Then, in the sharing step, for each $j = 1, \dots, k$, the simulator proceeds as follows.

1. For $i = 1, \dots, t$, sample $f^{(j)}(i) \leftarrow \mathbb{Z}_q$ and set $F_i^{(j)} = g^{f^{(j)}(i)}$, $y_i^{(j)} = \mathbf{pk}_i^{f^{(j)}(i)}$.
2. For $i = t+1, \dots, t+\ell$, sample $F_i^{(j)} \leftarrow \mathbb{G}$ randomly, with unknown, but implied discrete log $f^{(j)}(i)$. Now, $F_1^{(j)}, \dots, F_{t+\ell}^{(j)}$ implicitly define a the polynomial $f^{(j)}(x) \in \mathbb{Z}_q[X]_{t+\ell-1}$ in the exponent of g .
3. For $i = t+\ell+1, \dots, n$, compute $F_i^{(j)} = g^{f^{(j)}(i)}$ via Lagrange interpolation in the exponent from $F_1^{(j)}, \dots, F_{t+\ell}^{(j)}$.
4. For $i = t+1, \dots, n$, further compute $y_i^{(j)} = (F_i^{(j)})^{s_i}$.

Finally, the simulator generates a simulated transcript for the proof system as follows. For any challenge d :

1. Sample $z'(x) \leftarrow \mathbb{Z}_q[x]_{t+\ell-1}$.
2. For $i = 1, \dots, n$, set $\Gamma'_i = g^{z'(i)} / \prod_{j=1}^k (y_i^{(j)})^{d^j}$.
3. Reprogram the $\mathcal{RO}$ to yield $d \leftarrow \mathcal{RO}(\{y_1^{(j)}, \dots, y_n^{(j)}\}_{j=1}^k, \Gamma'_1, \dots, \Gamma'_n)$.

In the end, the simulator publishes $\{y_1^{(j)}, \dots, y_n^{(j)}\}_{j=1}^k$ as well as the simulated proof $\pi_{share} = (d, z(x))$.

We prove indistinguishability of our simulation from a real execution of the protocol. We first note that the distribution of $F_i^{(j)}$ and $y_i^{(j)}$ are exactly as in the real protocol. This follows from the fact that the $F_i^{(j)}$ are constructed via Lagrange interpolation and thus truly define an instance of $R_{\text{PDL}}^{\text{Mul}}$. As a consequence, this distribution holds for $y_i^{(j)}$ as well, which are computed as $(F_i^{(j)})^{s_i} = g^{s_i f^{(j)}(i)} = \text{pk}_i^{f^{(j)}(i)}$, and therefore the same as in the real protocol, from correctly distributed $F_i^{(j)}$. This also implies that the decryption proof in the reconstruction step can be executed exactly as in the real protocol for the simulated parties $i = t+1, \dots, n$. We are left to prove that our simulation of the NIZK is indistinguishable from a real execution. This argument is analogous to the HVZK proof of Theorem 4: since the polynomial $z'(x)$ is sampled randomly, the Γ'_i will also be random values in $\mathcal{G}$, exactly as in the real execution of the protocol. This concludes our proof. $\square$